

A Reproducible Computer Vision Study Across Fine-Grained Recognition, Pet Segmentation, and Aerial Object Detection

Yuheng Li

Student ID: 23307130334

GitHub: https://github.com/yuheng-li-ai/CV_Midterm

Model weights: [ModelScope repository](#)

Abstract

This report presents a unified experimental study for three computer vision tasks: Oxford-IIIT Pet image classification, Oxford-IIIT Pet semantic segmentation, and VisDrone aerial object detection with video multi-object tracking. A configuration-driven harness was implemented before large-scale training, so that all models, losses, metrics, checkpoints, and visualizations follow the same reproducible convention. For classification, ImageNet pretraining was the dominant factor, improving ResNet-18 test accuracy from 40.15% to about 89.40%, while the best extension was a pretrained ResNet-34 with 90.60% test accuracy. For segmentation, a from-scratch U-Net trained with Dice loss achieved the best required result, reaching 77.90% test mIoU. For detection, YOLO11m at 960-pixel input resolution substantially improved VisDrone mAP50-95 over YOLOv8n and YOLOv8s, and the resulting detector was used for tracking, occlusion analysis, and line-crossing counting on a 30-second rainy traffic video. Beyond reporting scores, the analysis emphasizes negative results, failure cases, and the relationship between metric gains and qualitative behavior.

1 Introduction

The assignment contains three tasks that stress different levels of visual understanding. Fine-grained pet classification tests instance-level appearance recognition. Pet semantic segmentation tests dense pixel-level localization. VisDrone detection and tracking test small-object perception in crowded aerial or high-view scenes. Treating these tasks as isolated scripts would make the experiments difficult to audit, so I first built a unified harness around configuration files, fixed output directories, checkpointing, CSV logging, curve generation, and report-ready visualization.

The main goal of this report is not only to obtain competitive numbers under the course setting, but also to explain why certain design choices worked or failed. Therefore, every required experiment is paired with at least one extension: pre-training and architecture capacity for classification, loss engineering for segmentation, and detector capacity, image resolution, tracker choice, and confidence threshold for detection and tracking.

2 Unified Experimental Harness

The project is organized around a single Python package, `cvhw2`. The command-line interface exposes five major commands: classification training, segmentation training, VisDrone conversion, detection training, and video tracking. Configurations are stored in `configs/`; scalar logs are written to `runs/`; checkpoints are written to `checkpoints/`; and final visual evidence is collected in `assets/`. This structure was useful because the assignment required many comparisons, and accidental changes in data loading, evaluation split, or checkpoint selection would otherwise make the results hard to interpret.

The released final and auxiliary checkpoints are hosted on ModelScope at <https://www.modelscope.cn/models/yuhengli/cv-hw2-final-model>. The ModelScope repository includes the three final models, the comparison checkpoints used in the report, the training configurations, and this final PDF.

The most important harness rule is validation-based model selection. During training, the best checkpoint is selected by validation accuracy for classification, validation mIoU for segmentation, and validation mAP50-95 for detection. The final test result is then computed from that best checkpoint instead of from the last epoch. This matters because several curves peaked before the final epoch, especially the pretrained ResNet-18 classification baseline and some segmentation losses. The harness also automatically saves learning curves and qualitative outputs, which prevents the final report from relying only on scalar metrics.

3 Task 1: Pet Image Classification

3.1 Dataset and Protocol

The classification task uses the Oxford-IIIT Pet dataset with 37 pet categories. Images are resized to 224×224 for full experiments. Training uses the official trainval portion with an internal validation split, and final evaluation uses the test split. The metric is top-1 accuracy. All main experiments use AdamW with weight decay 10^{-4} and a 20-epoch budget. For pretrained models, the backbone learning rate is 10^{-4} and the newly initialized classifier head learning rate is 10^{-3} . This split learning-rate design reflects the assumption that generic

ImageNet features should be adapted gently, while the task-specific 37-way classifier must learn faster.

3.2 Required Models

The required baseline is an ImageNet-pretrained ResNet-18 with a replaced final classifier. The pretraining ablation uses the same architecture but random initialization. The attention extension inserts a squeeze-and-excitation module after the final residual stage of a pretrained ResNet-18. These three experiments isolate three questions: whether transfer learning is necessary, whether the baseline is already strong, and whether a lightweight channel-attention module adds value.

Table 1: Classification results on Oxford-IIIT Pet.

Experiment	Best Val	Test Acc.	Note
ResNet-18 pretrained	0.9185	0.8940	required
ResNet-18 random	0.4475	0.4015	required
SE-ResNet-18	0.9112	0.8934	required
ResNet-34 pretrained	0.9330	0.9060	extension
Label smoothing	0.9130	0.8970	extension
Freeze then unfreeze	0.9239	0.8986	extension

3.3 Classification Results and Discussion

Table 1 shows that ImageNet pretraining is the strongest factor in this task. The pretrained ResNet-18 reached 89.40% test accuracy, while the same architecture trained from scratch reached only 40.15% under the same 20-epoch budget. The gap is too large to be explained by random variation. It indicates that generic low-level and mid-level features learned from ImageNet transfer well to pet breed recognition, especially when the target dataset is not large enough to train a deep residual network from scratch efficiently.

The random model was still improving at epoch 20, so the comparison should not be interpreted as proof that random initialization can never work. Rather, it shows that pretraining is vastly more sample- and compute-efficient under the assignment budget. This distinction is important: the ablation answers a practical question about the chosen training regime, not a universal theoretical question.

The SE-ResNet-18 result is a useful negative result. Although channel attention is often expected to help fine-grained recognition, the SE variant slightly underperformed the simpler pretrained ResNet-18. One plausible explanation is that the pretrained ResNet-18 already contains strong channel representations, while the newly inserted SE block introduces extra randomly initialized parameters that must be learned from limited data. The result does not invalidate attention mechanisms in general, but it warns against assuming that a more complex module will automatically improve a strong pretrained baseline.

Among the extensions, the pretrained ResNet-34 achieved the best test accuracy, 90.60%. This suggests that moderate additional depth can improve fine-grained discrimination when pretrained features are available. Label smoothing and

freeze-then-unfreeze gave smaller gains over the baseline. Label smoothing slightly improved test accuracy but lowered best validation accuracy, which suggests mild regularization rather than a decisive improvement. Freeze-then-unfreeze improved validation accuracy and test accuracy modestly, supporting the idea that stabilizing the new classifier head before full fine-tuning can reduce early optimization noise.

3.4 Classification Hyperparameter Grid Search

To address the hyperparameter-analysis requirement explicitly, I added a controlled grid search around the pretrained ResNet-18 baseline. The model architecture, data split, input size, batch size, backbone learning rate, and weight decay were held fixed. The search varied two optimization hyperparameters: the classifier-head learning rate and label smoothing. Each setting was trained for 8 epochs as a budgeted search, and validation accuracy was used as the selection criterion. The final test split was not used to choose the hyperparameter setting.

Table 2: Budgeted classification hyperparameter grid search. The best validation setting is highlighted.

Label smoothing	Head LR	Best epoch	Best Val	Test Acc.
0.00	5×10^{-4}	3	0.9112	0.8803
0.00	1×10^{-3}	3	0.9130	0.8760
0.00	2×10^{-3}	5	0.9130	0.8880
0.05	5×10^{-4}	5	0.9058	0.8863
0.05	1×10^{-3}	5	0.9167	0.8880
0.05	2×10^{-3}	7	0.9094	0.8926
0.10	5×10^{-4}	7	0.9076	0.8872
0.10	1×10^{-3}	6	0.9130	0.8902
0.10	2×10^{-3}	7	0.9076	0.8937

The grid search shows that the classification result is not highly sensitive to these two hyperparameters within the tested range. The best validation result, 0.9167, occurs at label smoothing 0.05 and head learning rate 10^{-3} , which is close to the original baseline design. Increasing label smoothing to 0.10 does not improve validation accuracy, suggesting that too much target softening can weaken the fine-grained breed signal. A larger head learning rate improves some test numbers but does not consistently improve validation performance, so I did not use it to redefine the final model. This supports a conservative conclusion: careful fine-tuning matters, but pre-training and backbone capacity remain the dominant factors in this task.

3.5 Error Analysis

The confusion matrix in Fig. 6 and the mistake grid in Fig. 7 show that most errors are semantically plausible rather than arbitrary. The top confusion pairs include American Pit Bull Terrier versus Staffordshire Bull Terrier, Staffordshire Bull Terrier versus American Pit Bull Terrier, Ragdoll versus Birman, American Pit Bull Terrier versus American Bulldog, and Egyptian Mau versus Bengal. These are visually similar classes with shared body shape, fur color, or facial structure.

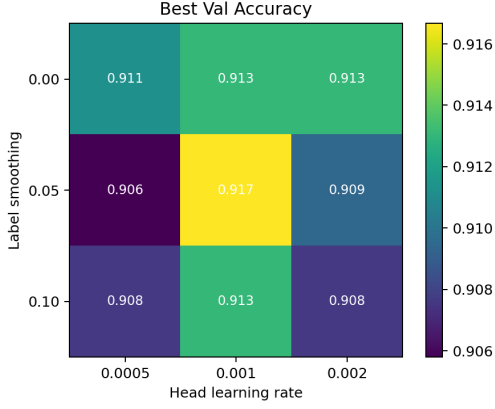


Figure 1: Validation accuracy heatmap for the classification hyperparameter grid. The narrow range of values indicates mild sensitivity compared with the much larger effect of ImageNet pretraining.

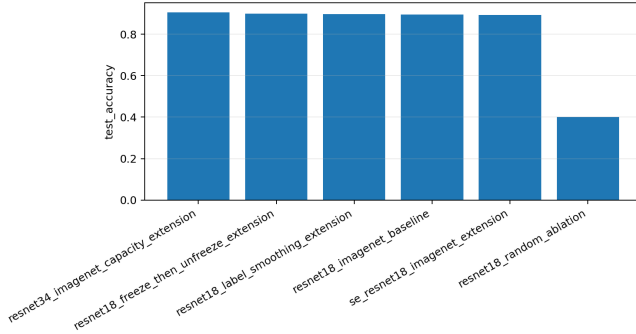


Figure 2: Classification test accuracy comparison. The main gain comes from ImageNet pretraining, while the best extension is the larger pretrained ResNet-34.

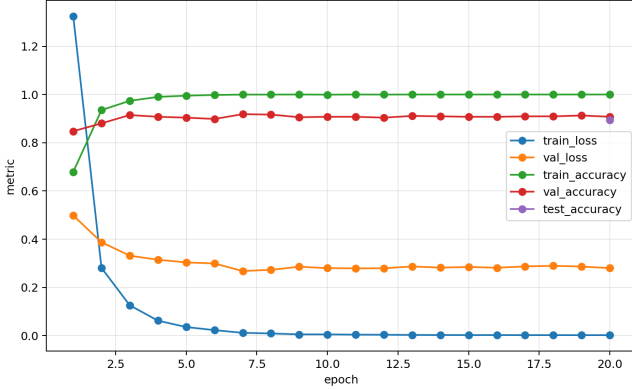


Figure 3: Training curves for the ImageNet-pretrained ResNet-18 baseline. The validation peak occurs early, showing why best-checkpoint evaluation is preferable to last-epoch reporting.

The model therefore fails mainly in fine-grained discrimination, not in coarse pet recognition.

This observation changes how the classification result should be judged. A test accuracy near 90% is strong, but the remaining errors are concentrated exactly where subtle anatomical cues are required. More data augmentation, higher-resolution crops, or part-aware features might help these cases more than a generic increase in classifier capacity.

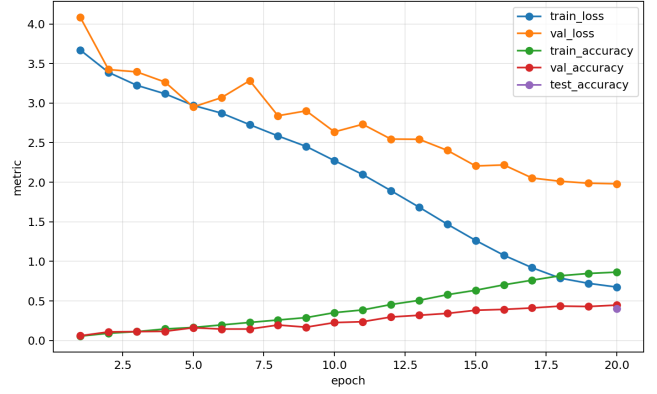


Figure 4: Training curves for the randomly initialized ResNet-18. Compared with Fig. 3, convergence is much slower and the validation accuracy remains far below the pretrained model under the same epoch budget.

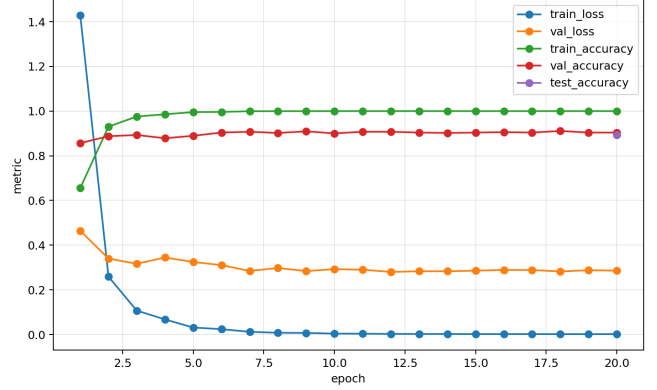


Figure 5: Training curves for the SE-ResNet-18 extension. The curve shows that attention did not create a clear generalization gain over the simpler pre-trained ResNet-18 baseline.

4 Task 3: Pet Semantic Segmentation

4.1 Dataset, Model, and Losses

The segmentation task also uses Oxford-IIIT Pet, but with trimap masks converted into a three-class semantic segmentation problem: pet foreground, boundary, and background. The model is a manually implemented U-Net trained from random initialization. It contains four encoder stages, a bottleneck, four decoder stages, transposed-convolution upsampling, and skip concatenations. The main metric is mean Intersection over Union (mIoU).

Three losses form the required comparison. Cross-entropy treats each pixel independently and is stable early in training. Multiclass Dice loss optimizes region overlap more directly. The combined CE+Dice objective attempts to balance stable pixel-wise supervision with overlap-oriented optimization. For a class c , the Dice term is computed as

$$\mathcal{L}_{Dice} = 1 - \frac{1}{C} \sum_{c=1}^C \frac{2 \sum_i p_{i,c} y_{i,c} + s}{\sum_i p_{i,c} + \sum_i y_{i,c} + s}, \quad (1)$$

where $p_{i,c}$ is the predicted probability, $y_{i,c}$ is the one-hot target, $C = 3$, and s is the smoothing constant. All main segmentation experiments use image size 224, batch size 16, AdamW,

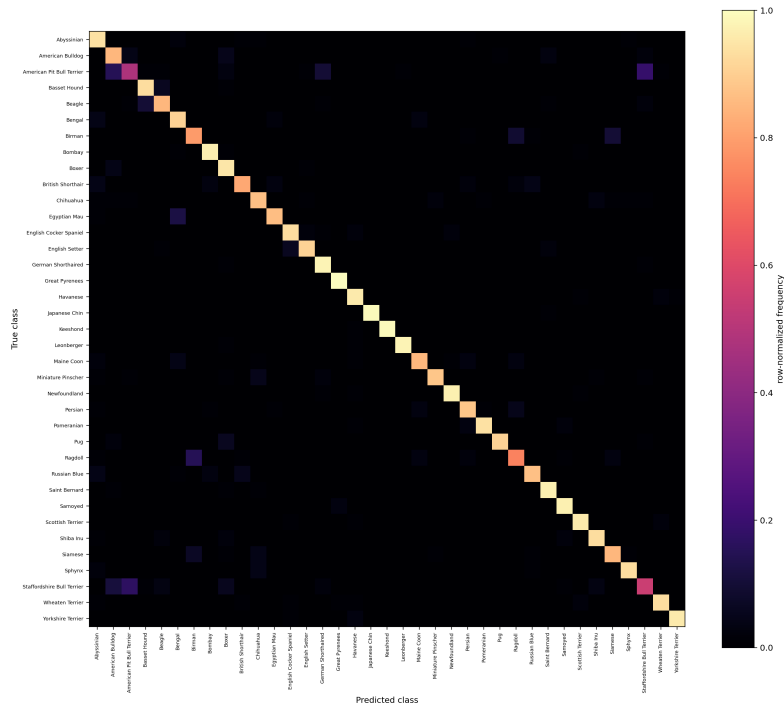


Figure 6: Confusion matrix for the pretrained ResNet-18 baseline. The matrix is dominated by a strong diagonal consistent with the near-90% test accuracy. The visible off-diagonal blocks correspond to visually similar breeds (e.g., pit bull / Staffordshire / bulldog; Ragdoll / Birman / Siamese) rather than random category failures, so the model fails at fine-grained breed cues such as head shape and coat pattern rather than at coarse pet recognition.

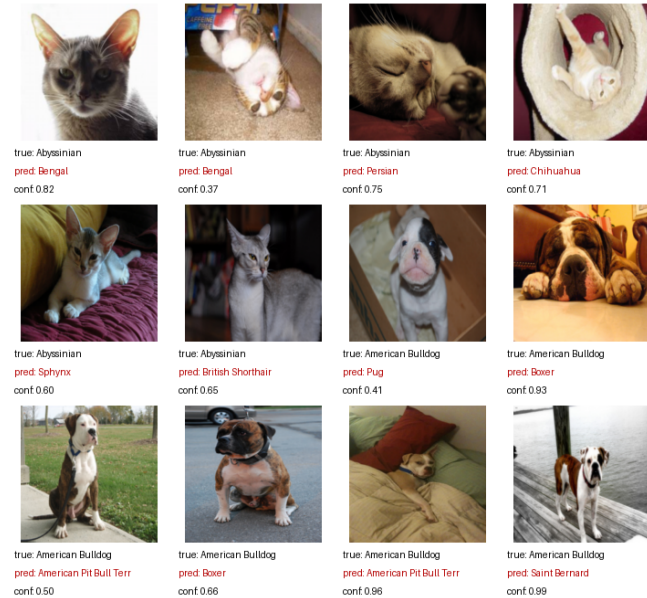


Figure 7: Representative classification mistakes from the pretrained ResNet-18 baseline. Most wrong predictions are not caused by background clutter; the animals are clearly visible, but discriminative breed cues are subtle. The pit bull / Staffordshire / bulldog group shares short fur and head structure, while Ragdoll / Birman / Siamese share coat color and facial masks.

4.2 Required Loss Comparison

Table 3: Segmentation results for the 60-epoch U-Net loss comparison.

Loss	Best Epoch	Best Val mIoU	Test mIoU
Cross-Entropy	42	0.7640	0.7640
Dice	54	0.7748	0.7790
CE + Dice	37	0.7681	0.7741

Dice loss achieved the best required result, with 77.90% test mIoU. This supports the hypothesis that overlap-oriented supervision is better aligned with the mIoU metric than pure pixel-wise cross-entropy. Cross-entropy remained a strong baseline, but it peaked earlier and did not match the final overlap quality of Dice. CE+Dice improved over CE but did not surpass Dice alone, suggesting that the stabilizing CE term was not necessary once the training budget was long enough for Dice optimization to become stable.

The interpretation should remain bounded. Dice was best in this controlled setup, but this does not mean Dice is universally superior. Cross-entropy can be easier to optimize early, and Dice can be sensitive when classes are extremely small or predictions are unstable. In this dataset and schedule, however, the object-overlap objective matched the evaluation target well.

4.3 Loss Engineering Extensions

Two additional loss experiments were conducted. First, the Dice smoothing constant was reduced from 1.0 to 0.1. This slightly increased best validation mIoU from 0.7748 to 0.7769,

cosine learning-rate scheduling, and a 60-epoch budget.

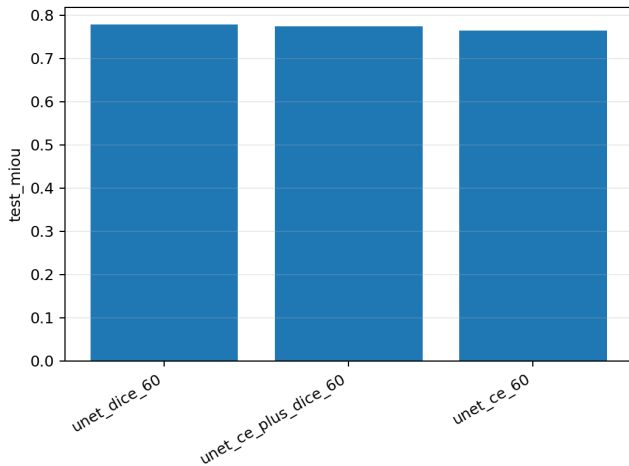


Figure 8: Test mIoU for the required 60-epoch segmentation losses. Dice loss gives the strongest required result.

but the test mIoU was almost unchanged, 0.7785 versus 0.7790. The smaller smoothing value may make the objective more sensitive to foreground and boundary errors, but the difference is too small to claim a robust generalization improvement.

Second, Focal Loss was tested as a hard-pixel emphasis strategy. It reached 0.7645 test mIoU, close to cross-entropy but below Dice. This is another useful negative result. Boundary pixels in trimap segmentation are hard not only because they are informative, but also because they are ambiguous and sometimes visually noisy. Emphasizing all hard pixels can therefore over-focus the model on uncertain regions instead of improving structural mask quality.

Table 4: Segmentation extension results.

Experiment	Best Epoch	Best Val mIoU	Test mIoU
Dice, smooth 1.0	54	0.7748	0.7790
Dice, smooth 0.1	55	0.7769	0.7785
Focal Loss	45	0.7637	0.7645

4.4 Qualitative Segmentation Analysis

Figure 12 compares ground-truth masks, predictions, binary error maps, and overlays for representative difficult examples. The dominant errors are not complete object misses; they are boundary and contour errors. Thin legs, ears, fur edges, and low-contrast foreground-background transitions are harder than the main body region. This explains why mIoU can improve while the masks still look imperfect around fine structures.

This qualitative evidence also clarifies the Focal Loss result. If the difficult pixels are mainly ambiguous boundaries, then increasing their weight does not necessarily create cleaner boundaries. It may instead amplify label noise or local uncertainty. Dice improves global overlap, but even it does not fully solve contour precision. A natural next step would be a boundary-aware loss or higher-resolution decoding, but that is beyond the required scope.

5 Task 2: VisDrone Detection and Tracking

5.1 VisDrone Data Preparation

The detection task uses the public VisDrone2019-DET dataset rather than a substitute dataset. The raw train, validation, and test-dev splits were downloaded and converted into YOLO format. The converted dataset contains 6471 training images, 548 validation images, and 1610 test-dev images. The training labels contain 343205 valid boxes after ignoring VisDrone non-target categories. The 10 object classes are pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, and motor.

VisDrone is substantially harder than ordinary object detection datasets because many targets are small, partially occluded, densely packed, or viewed from high altitude. Therefore, modest mAP values are expected, especially for lightweight detectors at 640-pixel resolution.

5.2 Detector Training Results

Table 5: VisDrone detector comparison.

Model	Size	Epoch	Prec.	Rec.	mAP50-95
YOLOv8n	640	48	0.4188	0.3226	0.1665
YOLOv8s	640	47	0.5187	0.3922	0.2211
YOLO11m	960	36	0.6240	0.5257	0.3310

YOLOv8n is the required lightweight one-stage baseline. It reached 0.2985 mAP50 and 0.1665 mAP50-95. YOLOv8s improved mAP50-95 to 0.2211, showing that extra capacity helps in dense aerial scenes. The strongest extension was YOLO11m trained at image size 960. Even though the run was manually stopped at epoch 36 because of deadline constraints, it reached 0.5356 mAP50 and 0.3310 mAP50-95, clearly outperforming YOLOv8s.

The improvement has two sources that cannot be perfectly separated in this experiment: the newer YOLO11m architecture and the larger 960-pixel input size. Both are plausible contributors. Small objects occupy very few pixels in VisDrone, so higher resolution directly improves feature visibility. At the same time, the larger model has more capacity to represent dense object patterns and difficult class distinctions. The practical conclusion is that for VisDrone, a stronger detector at higher resolution is much more useful for downstream tracking than a very small detector, as long as the available GPU memory permits it.

A second pattern in Table 5 deserves a structural explanation: precision is systematically higher than recall for all three detectors, with the gap roughly preserved around 0.10 (YOLOv8n: 0.42 vs 0.32; YOLOv8s: 0.52 vs 0.39; YOLO11m: 0.62 vs 0.53). The asymmetry is not a sign of insufficient training. Precision is the fraction of predicted boxes that are correct, so its denominator only contains predictions the detector was confident enough to emit; the lowest-

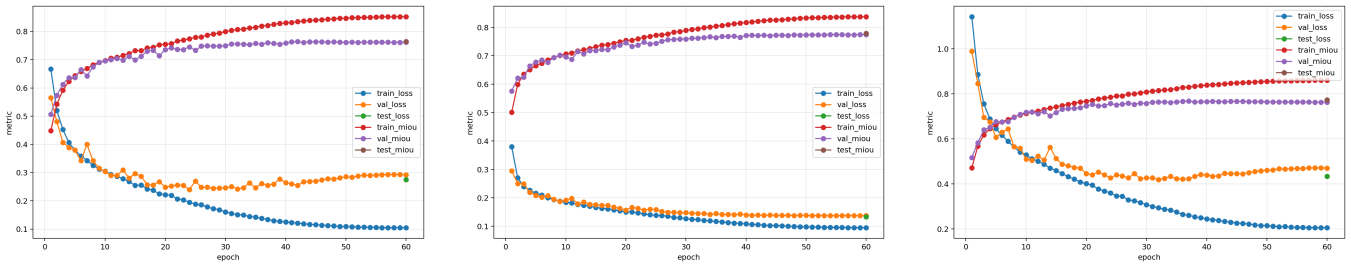


Figure 9: Training curves for the required segmentation losses: CE, Dice, and CE+Dice from left to right. These panels have the same aspect ratio and are therefore compared side by side. Dice improves more slowly but reaches the best validation and test mIoU.

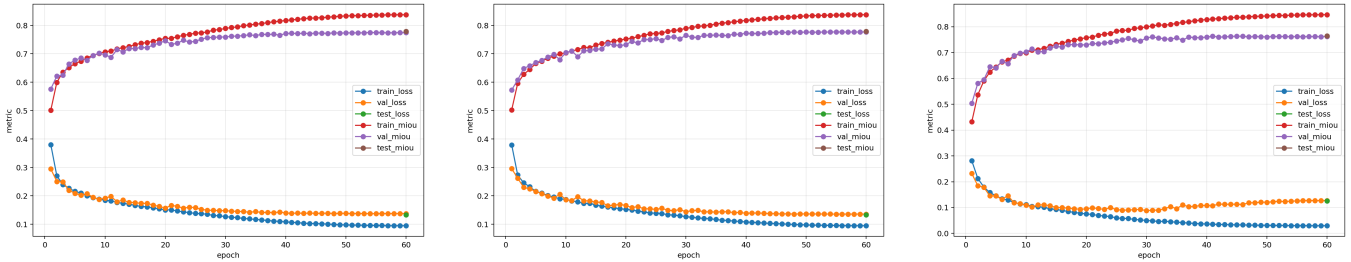


Figure 10: Training curves for segmentation extensions: Dice with smooth 1.0, Dice with smooth 0.1, and Focal Loss from left to right. The two Dice variants are close, while Focal Loss does not improve mIoU despite emphasizing hard pixels.

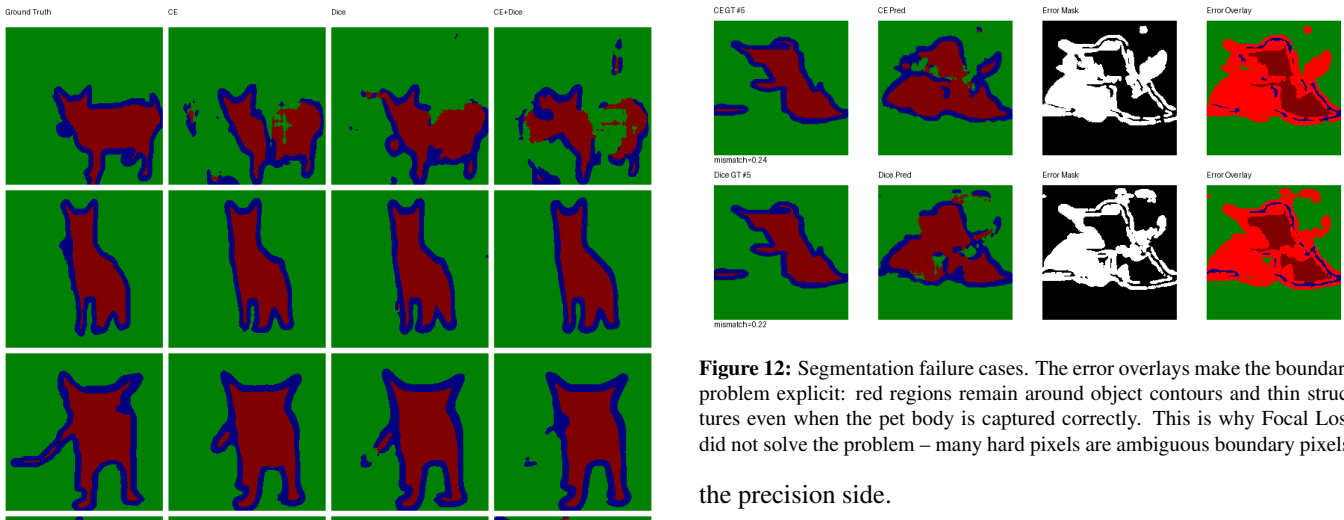


Figure 11: Compact qualitative mask examples for the required segmentation losses (columns: ground truth, CE, Dice, CE+Dice). Dice improves global object overlap while CE remains less directly aligned with overlap. Remaining errors concentrate on ears, legs, fur boundaries, and regions where pet and background share color.

confidence candidates are filtered out before the metric is computed, biasing the denominator toward easier cases. Recall is the fraction of ground-truth boxes that were found, so its denominator contains every annotated object, including the long tail of small, occluded, or low-contrast targets that the detector physically cannot resolve at the chosen input resolution. Recall is therefore bounded from above by what the network can perceive, while precision is bounded by how often the network is willing to fire, and the gap persists even when the model becomes substantially stronger. The resolution lift from 640 to 960 closed more of the gap than capacity alone because it directly addresses the recall ceiling on tiny objects rather than

Figure 12: Segmentation failure cases. The error overlays make the boundary problem explicit: red regions remain around object contours and thin structures even when the pet body is captured correctly. This is why Focal Loss did not solve the problem – many hard pixels are ambiguous boundary pixels.

the precision side.

This asymmetry also has a favorable consequence for the downstream tracking pipeline. A precision-leaning detector emits fewer false-positive boxes, which would otherwise be assigned new track IDs by ByteTrack or BoT-SORT and inflate the line-crossing count through ghost trajectories. A missed detection, by contrast, can often be bridged across one or two frames by the tracker’s Kalman prediction without permanently breaking the ID. Therefore, when the available recall is reasonable, the precision side being higher is a desirable operating point for counting; the lightweight YOLOv8n baseline fails the pipeline mainly through insufficient recall on small targets, not through low precision.

5.3 Video Tracking Setup

The trained YOLO11m detector was used for a 30-second tracking and counting experiment. The original uploaded video was 1280×720 at 30 FPS and 41.2 seconds long. The

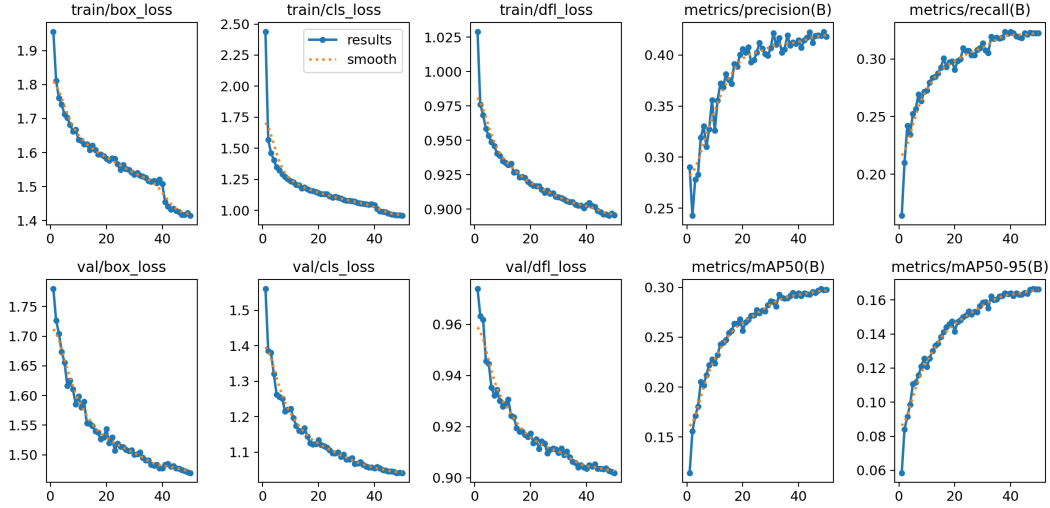


Figure 13: YOLOv8n training summary on VisDrone. Precision and recall climb together, but precision stays consistently above recall throughout training because the long tail of tiny, occluded VisDrone targets is mostly unrecoverable at 640-pixel input. Recall plateaus near 0.33, leaving the lightweight model unsuitable for reliable downstream counting.

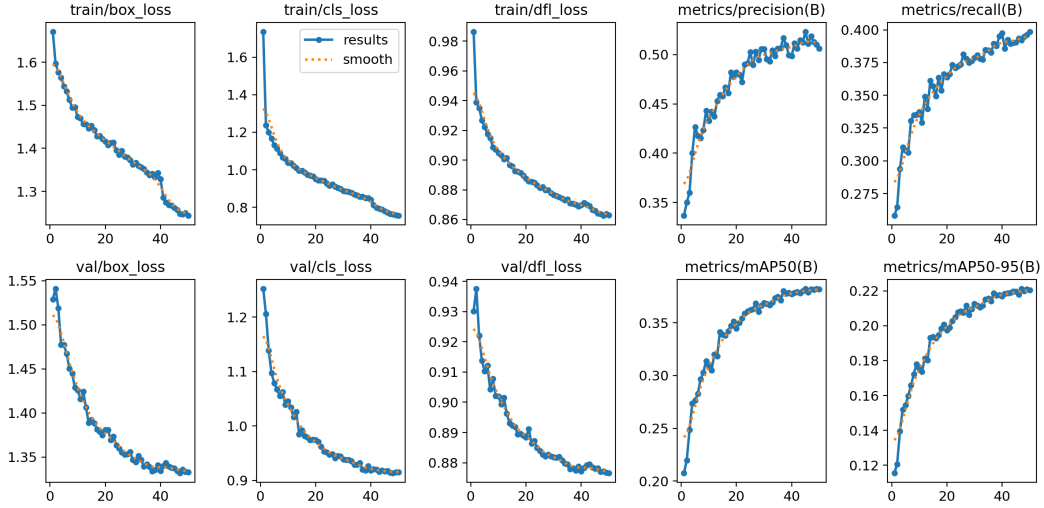


Figure 14: YOLOv8s training summary on VisDrone. Compared with YOLOv8n, the larger backbone lifts precision, recall, mAP50, and mAP50-95 in parallel, but the precision–recall gap stays roughly the same: the low baseline score was a capacity limit, not a training failure. The trade-off is model size and training time, which matters because the detector is later reused for video tracking.

final clip uses the 10s–40s segment, giving exactly 900 frames. The scene contains rainy road traffic, pedestrians, bicycles, e-bikes, motorcycles, cars, umbrellas, reflections, dense crossing motion, and partial occlusions. This makes it more challenging and more realistic than a sparse clean-weather video.

The virtual counting line is placed from (250, 520) to (1020, 520) across the lower road region. A crossing is counted when the center of a tracked bounding box changes sign relative to this line. Each track ID is counted at most once, which prevents repeated counting of a stable track. However, if an ID switch occurs before and after the line, duplicate counting can still happen. The current logic counts total crossings and does not separate direction.

Table 6: Tracking and line-crossing results on the 30-second video.

Tracker	Conf.	Unique IDs	Avg. Obj.	Count
ByteTrack	0.25	183	5.79	11
BoT-SORT	0.25	177	5.90	8
ByteTrack	0.50	86	2.30	5

5.4 Tracking Results and Interpretation

Table 6 shows that ByteTrack at confidence 0.25 produced the main result, with 183 unique track IDs, an average of 5.79 tracked objects per frame, and 11 line crossings. BoT-SORT at the same threshold was more conservative and counted 8 crossings. ByteTrack at confidence 0.50 counted only 5 crossings because many weak detections were suppressed.

This comparison illustrates a precision-recall trade-off at

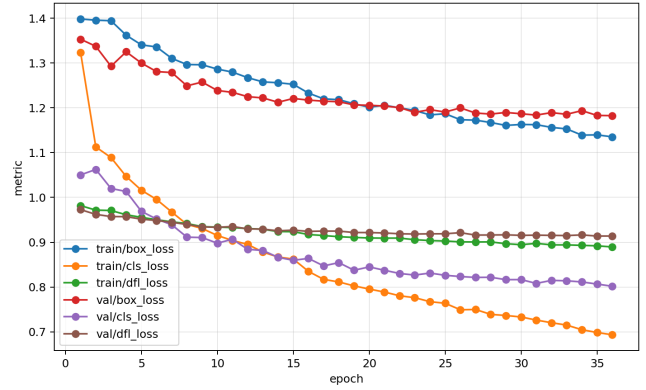
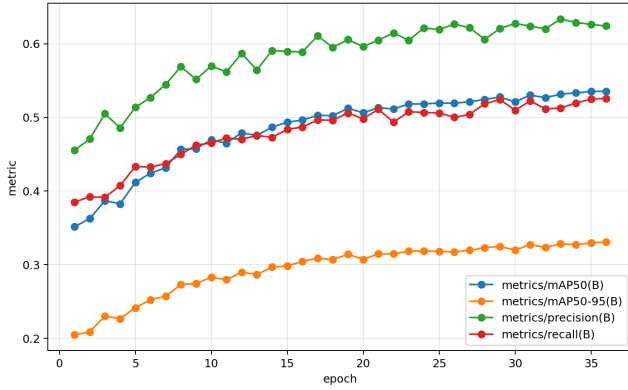


Figure 15: YOLO11m validation metrics and losses on VisDrone. The manually truncated run still achieved the best recorded score at epoch 36, while the losses were still decreasing slowly, suggesting possible remaining improvement with more epochs.

the tracking level. A lower confidence threshold keeps weak detections and reduces missed crossings, but it can also introduce short-lived tracks and more opportunities for ID fragmentation. A higher threshold produces cleaner detections but misses small, blurred, or occluded road users, which directly causes under-counting. In this video, the 0.25 threshold is more appropriate because the scene contains rain, small targets, and partial occlusion.

The difference between ByteTrack and BoT-SORT should not be simplified as one being universally better. The two trackers made different association decisions in dense traffic. ByteTrack preserved more crossing candidates and therefore produced a higher count. BoT-SORT was more conservative in the final count. Without manually annotated video ground truth, the count should be treated as a reproducible estimate rather than an absolute traffic measurement.

5.5 Counting Error Analysis

The line-crossing module is intentionally simple and transparent. Its main strength is that it uses persistent track IDs, so a stable object is not counted repeatedly after crossing the line. Its main weakness is that it inherits all detector and tracker errors. Under-counting occurs when a target is not detected near the line, when confidence thresholding suppresses it, or when the track disappears before the sign change is observed. Duplicate counting can occur if one physical object receives a new ID after an occlusion and crosses the line again under that new ID. Perspective also matters: a 2D image line is only an approximation of a real-world road boundary, especially when the camera is high and tilted.

Rain and wet-road reflections made this video a better stress test than a clean static scene. The final results are therefore realistic but imperfect. They demonstrate the full required pipeline—detection, tracking IDs, occlusion frames, and line counting—while also making clear that reliable traffic statistics would require annotated video ground truth, camera calibration, and direction-aware counting.

6 Cross-Task Reflection

Across the three tasks, the most consistent lesson is that stronger methods help only when they match the dominant bottleneck. In classification, the bottleneck is fine-grained representation, so ImageNet pretraining and moderate depth are useful. Adding SE attention without enough evidence did not help. In segmentation, the bottleneck is region overlap and boundary quality, so Dice loss improves mIoU, but Focal Loss does not solve ambiguous trimap boundaries. In VisDrone detection, the bottleneck is small-object visibility and dense localization, so a larger detector and higher input resolution substantially improve mAP and downstream tracking.

The negative results are as important as the positive ones. Random initialization was far worse than pretraining; SE attention did not beat the simpler baseline; Focal Loss did not beat Dice; and a higher tracking confidence threshold reduced the crossing count. These results prevent the report from becoming a list of successful tricks. They show that each modification must be judged by the task metric, qualitative evidence, and failure mode.

7 Conclusion

This project completed the required image recognition, semantic segmentation, object detection, video tracking, occlusion analysis, and line-crossing counting tasks using a unified reproducible harness. The best classification model was a pre-trained ResNet-34 with 90.60% test accuracy. The best required segmentation model was a from-scratch U-Net with Dice loss, reaching 77.90% test mIoU. The strongest VisDrone detector was YOLO11m at 960-pixel resolution, reaching 0.5356 mAP50 and 0.3310 mAP50-95 before manual truncation at epoch 36. The tracking stage showed that ByteTrack with a lower confidence threshold produced the most complete crossing count, while higher thresholds under-counted difficult objects.

The main limitation is that some extensions were constrained by time and compute. The YOLO11m run may have improved with a full 50-epoch schedule, and the video count-



Figure 16: Four consecutive dense and occluded tracking frames. The scene contains overlapping pedestrians, bicycles or e-bikes, vehicles, rain, and small objects, making ID preservation difficult.



Figure 17: A consecutive line-crossing sequence. The tracked object crosses the virtual line and the count changes from 2 to 3 around frame 632.

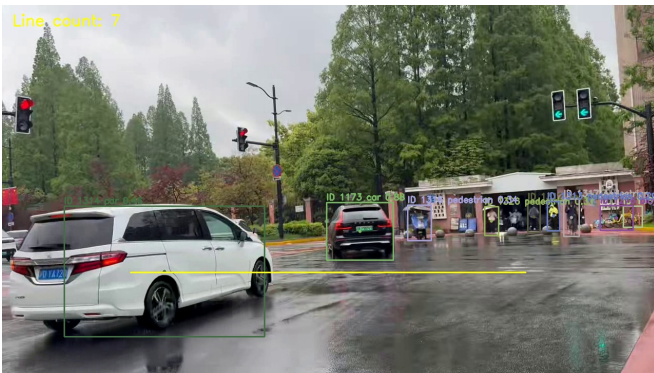


Figure 18: A later tracking snapshot with the accumulated line count visible in the top-left overlay. Some visible road users remain difficult under rain, distance, and occlusion, so the count should be treated as a reproducible algorithmic estimate rather than a manually verified traffic statistic.

ing result lacks manually annotated ground truth. Nevertheless, the experiments provide quantitative comparisons, visual evidence, and failure analysis for all required components, while preserving reproducibility through the harness and configuration files.

A Appendix A: Reproducibility and Harness Details

A.1 Configuration-Driven Experiment Control

The harness was designed before the main experiments because the assignment contains three visually different tasks but still requires fair comparison. A direct collection of ad-hoc notebooks would make it easy to change the split, image size, augmentation, or checkpoint rule accidentally between runs. The project therefore uses a configuration-first interface. Each experiment is specified by a YAML file, and the command line only selects which configuration to run. This makes each result traceable to a concrete set of hyperparameters rather than to undocumented interactive choices.

For classification, the configuration controls the architecture, pretraining flag, optional attention module, image size,

batch size, validation ratio, optimizer, learning rates, weight decay, label smoothing, and checkpoint directory. For segmentation, it controls U-Net width, image size, loss function, Dice smoothing, batch size, and output visualization count. For detection, it controls YOLO model family, image size, batch size, worker count, pretrained weights, optimizer, learning rate, patience, and output directory. This separation between code and configuration is useful because it lets the report compare methods while keeping the data protocol fixed.

Table 7: Main reproducibility controls enforced by the harness.

Control	Purpose
Fixed random seed	Makes train/validation splitting and stochastic training behavior more reproducible.
Validation-best checkpoint	Prevents the final metric from depending on a lucky or unlucky last epoch.
CSV metric log	Keeps every epoch available for later curve plotting and diagnosis.
Separate run directories	Prevents baseline and extension artifacts from overwriting one another.
Config snapshot in checkpoint	Keeps model weights tied to the exact training configuration.
Report-ready assets	Ensures that figures are generated from saved metrics rather than manually edited.

The most important rule is validation-based selection. Classification chooses the checkpoint with the highest validation accuracy, segmentation chooses the highest validation mIoU, and detection uses the best validation mAP50-95 reported by the YOLO trainer. This rule is intentionally conservative. It prevents a stronger-looking test number from being selected after looking at the test split, and it also avoids reporting a late checkpoint that has already started to overfit. The rule is particularly visible in classification, where the pretrained ResNet-18 baseline reaches its validation peak early, while the randomly initialized model is still improving at the end of the fixed 20-epoch budget.

A.2 Why a Unified Harness Matters

The three tasks expose different failure modes, but they share the same experimental risks. In classification, the major risk is accidentally comparing a pretrained model with a randomly initialized model under different augmentation or learning-

rate schedules. In segmentation, the major risk is changing the mask resize operation or class mapping between losses. In detection, the risk is confusing VisDrone annotation conversion, image size, and model capacity. A unified harness does not remove all uncertainty, but it makes these assumptions explicit.

The harness also makes negative results useful. SE attention did not improve the pretrained ResNet-18 baseline, Focal Loss did not beat Dice loss, and a high tracking confidence threshold reduced the line-crossing count. These results would be difficult to trust if each experiment were executed manually with different hidden settings. Because the harness fixes data splits, metrics, and output rules, the negative results become interpretable evidence rather than random training noise.

A.3 Compute-Budget Interpretation

Several extensions were selected under a practical compute budget rather than an exhaustive research setting. The classification grid search used 8 epochs per setting instead of a full 20-epoch schedule. This was sufficient to compare local sensitivity to label smoothing and head learning rate, but it was not intended to replace the final 20-epoch ResNet-34 result. The YOLO11m detector was stopped at epoch 36 because the report deadline was more important than chasing a marginal final improvement. These choices should be interpreted as budgeted experimental design rather than as claims of global optimality.

B Appendix B: Classification Details Beyond the Main Text

B.1 Input Pipeline and Feature Dimensions

The classification input pipeline first resizes every RGB image to 224×224 . During training, horizontal flipping and color jitter are applied. Horizontal flipping tests whether the model can recognize a breed regardless of whether the animal faces left or right. Color jitter randomly perturbs brightness, contrast, and saturation, which reduces dependence on one lighting condition. The image is then converted to a tensor and normalized by ImageNet mean and standard deviation. This final normalization is not arbitrary: it aligns the target data distribution with the distribution expected by pretrained ResNet weights.

For a batch of B images, the tensor enters ResNet as $B \times 3 \times 224 \times 224$. The first convolution and max-pooling reduce the spatial size to 56×56 . The four residual stages then produce features with channel counts 64, 128, 256, and 512, while the final spatial map is 7×7 . Adaptive global average pooling converts the $B \times 512 \times 7 \times 7$ tensor into $B \times 512$. The original ImageNet classifier is replaced with a new linear layer whose output dimension is 37. These 37 raw scores are logits; the predicted class is the index of the largest logit.

This dimensional view explains why the classification head can learn quickly. Only the final 512×37 mapping is randomly

initialized, while most earlier filters already encode general visual primitives. The split learning-rate design follows this fact: the head uses a larger learning rate because it is new, while the backbone uses a smaller learning rate because its pretrained filters should be adapted but not destroyed.

B.2 Cross-Entropy and Label Smoothing

The ordinary cross-entropy loss for an example with true class y and predicted softmax probability p_y is

$$\mathcal{L}_{CE} = -\log p_y. \quad (2)$$

With a one-hot target, the correct class has target probability 1 and every other class has target probability 0. This is a strong training signal, but it can make the model over-confident, especially in fine-grained recognition where two labels may be visually close. Label smoothing replaces the hard target distribution with a softened target distribution:

$$q_k = \begin{cases} 1 - \epsilon, & k = y, \\ \epsilon / (K - 1), & k \neq y, \end{cases} \quad (3)$$

where $K = 37$ and ϵ is the smoothing value. The loss becomes

$$\mathcal{L}_{LS} = -\sum_{k=1}^K q_k \log p_k. \quad (4)$$

In this assignment, the original label-smoothing extension used $\epsilon = 0.1$. The later grid search tested $\epsilon \in \{0, 0.05, 0.10\}$ together with three head learning rates. The best validation point was $\epsilon = 0.05$ and head learning rate 10^{-3} . However, the grid did not produce a dramatic improvement. This supports the interpretation that label smoothing is a secondary regularizer rather than the main source of performance.

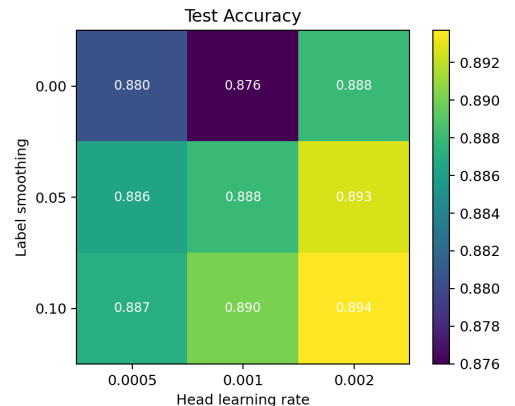


Figure 19: Test accuracy heatmap for the classification hyperparameter grid. The best test value in this short grid occurs at label smoothing 0.10 and head learning rate 2×10^{-3} , but model selection should follow validation accuracy, not the test split. The mismatch between validation-best and test-best is a useful reminder that test performance should not be optimized directly.

B.3 Why Pretraining Dominates the Grid

The grid search changed validation accuracy by roughly one percentage point. In contrast, ImageNet pretraining changed

test accuracy from 40.15% to about 89.40%. The difference in scale matters. Hyperparameters can polish a strong representation, but they cannot fully compensate for a weak representation under a limited training budget. A randomly initialized ResNet-18 has millions of parameters and must learn low-level edge detectors, mid-level textures, object parts, and breed-specific features from the Oxford-IIIT Pet training set alone. A pretrained ResNet-18 starts from a representation already shaped by large-scale natural-image data.

The failure cases also support this reading. The pretrained model is rarely confused about whether an image contains a pet. Its errors occur among visually similar breeds. This means the model has already solved the coarse recognition problem; remaining gains require better fine-grained cues. ResNet-34 helps because it increases backbone capacity while preserving the benefit of pretraining. SE attention, by contrast, adds randomly initialized channel reweighting on top of an already strong feature extractor. Without enough additional data, that new module does not automatically produce a cleaner breed representation.

C Appendix C: Segmentation Mathematics and Additional Evidence

C.1 IoU, Dice, and Their Relationship

For a binary class mask, let P be the predicted foreground set and G be the ground-truth foreground set. The Intersection over Union is

$$IoU(P, G) = \frac{|P \cap G|}{|P \cup G|} = \frac{|P \cap G|}{|P| + |G| - |P \cap G|}. \quad (5)$$

The Dice coefficient is

$$Dice(P, G) = \frac{2|P \cap G|}{|P| + |G|}. \quad (6)$$

If $I = |P \cap G|$ and $U = |P \cup G|$, then $|P| + |G| = U + I$. Therefore

$$Dice = \frac{2I}{U + I} = \frac{2(I/U)}{1 + (I/U)} = \frac{2IoU}{1 + IoU}. \quad (7)$$

This monotonic relationship means that maximizing Dice also tends to maximize IoU for a single class. For multiclass segmentation, the relationship is not perfectly identical because scores are averaged over classes and computed from soft probabilities during training, but the direction is still strongly aligned. This is why Dice loss is more directly related to mIoU than pixel-wise cross-entropy.

Cross-entropy asks whether each pixel is assigned the correct class. Dice asks whether the predicted region overlaps the true region. These are related but not identical objectives. A model can improve many easy background pixels under cross-entropy without significantly improving object overlap. In a trimap task, the boundary class is small and ambiguous, so overlap-based feedback is valuable. The observed result—Dice outperforming CE and CE+Dice—is therefore consistent with the metric geometry.

C.2 Smooth Dice and Numerical Stability

The implemented Dice loss uses a smoothing constant s :

$$Dice_c = \frac{2 \sum_i p_{i,c} y_{i,c} + s}{\sum_i p_{i,c} + \sum_i y_{i,c} + s}. \quad (8)$$

The smoothing term prevents numerical instability when a class is absent or nearly absent in a batch. A larger s also makes the fraction less sensitive when masks are small. The smooth-0.1 extension reduces this damping, making the objective respond more strongly to small overlap changes. In the results, smooth 0.1 slightly improved best validation mIoU but did not improve test mIoU. This suggests that the default smooth value was not the bottleneck.

C.3 Why Focal Loss Did Not Help

Focal Loss reweights cross-entropy by a factor based on prediction difficulty:

$$\mathcal{L}_{Focal} = -(1 - p_t)^\gamma \log(p_t), \quad (9)$$

where p_t is the predicted probability of the true class. Easy pixels with high p_t receive small weights, while hard pixels receive larger weights. This is attractive for imbalanced problems, but hard pixels are not always reliable pixels. In pet trimap segmentation, many hard pixels lie on fur boundaries, ears, legs, shadows, and low-contrast transitions. Some are genuinely ambiguous even to a human observer. Increasing their weight can focus learning on noisy boundaries rather than on stable region overlap.

This explains why Focal Loss was competitive with CE but below Dice. The segmentation problem is not merely a class-imbalance problem; it is an overlap and boundary-quality problem. Dice loss directly rewards global region overlap, while Focal Loss still relies on pixel-wise classification and only changes the weighting of pixels.

C.4 Boundary Ambiguity and Evaluation

The Oxford-IIIT Pet trimap annotations contain a boundary class that is visually thin and semantically ambiguous. A boundary pixel may lie near fur, shadow, motion blur, or background texture. When the image is resized to 224×224 , fine hair and edge details are further compressed. This makes perfect boundary agreement unrealistic. The qualitative results show that most large body regions are captured correctly, but errors remain along the contour. This is exactly the region where mIoU is sensitive: a thin boundary error can reduce the boundary-class IoU even if the pet body looks visually acceptable.

Future improvements should therefore target boundary modeling rather than only changing the global loss. Possible directions include boundary-aware auxiliary heads, higher input resolution, deep supervision at decoder stages, or a CRF-like post-processing step. These were not used because the

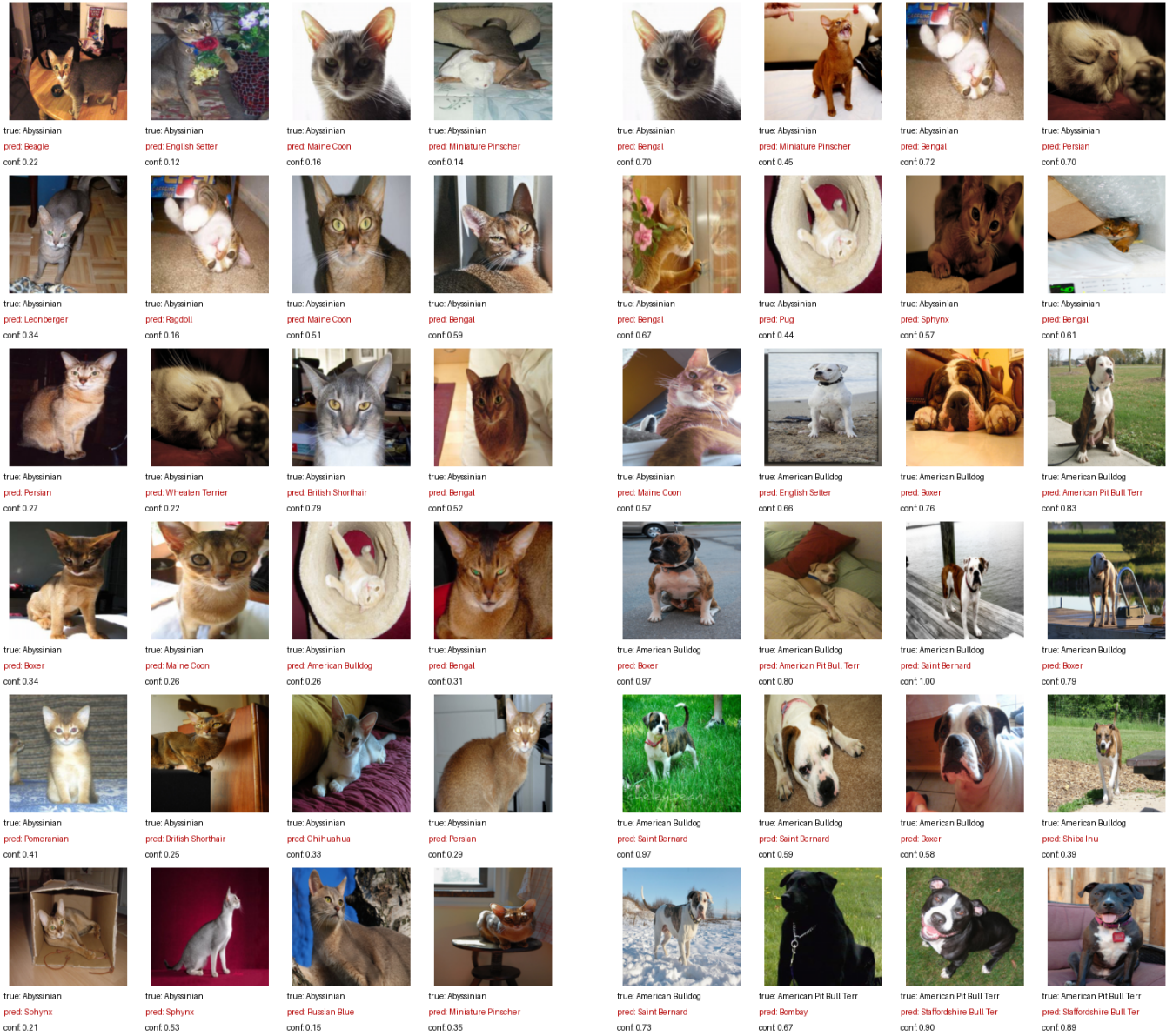


Figure 20: Additional classification error grids. Left: randomly initialized ResNet-18, which produces broader and less stable errors because it lacks transferable ImageNet features. Right: SE-ResNet-18, whose errors remain fine-grained despite adding attention. These qualitative grids explain why pretraining is a first-order factor while SE is not a guaranteed improvement.

assignment required a controlled U-Net comparison, and additional post-processing would make the loss comparison less clean.

D Appendix D: Detection Metrics and Deeper Interpretation

D.1 IoU and Matching

Object detection evaluation begins by matching predicted boxes to ground-truth boxes. For a predicted box B_p and a ground-truth box B_g , the intersection-over-union is

$$IoU(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \quad (10)$$

A prediction is counted as a true positive if it has the correct class, exceeds the IoU threshold, and is matched to an unused ground-truth box. Duplicate predictions for the same object become false positives after the first match. Ground-truth boxes without a matched prediction become false negatives. This matching rule is stricter than simply drawing plausible boxes; it requires both classification and localization.

At a fixed confidence threshold, precision and recall are

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}. \quad (11)$$

Average Precision is computed by sorting predictions by confidence and integrating the precision-recall curve. Conceptually,

$$AP = \int_0^1 P(R) dR, \quad (12)$$

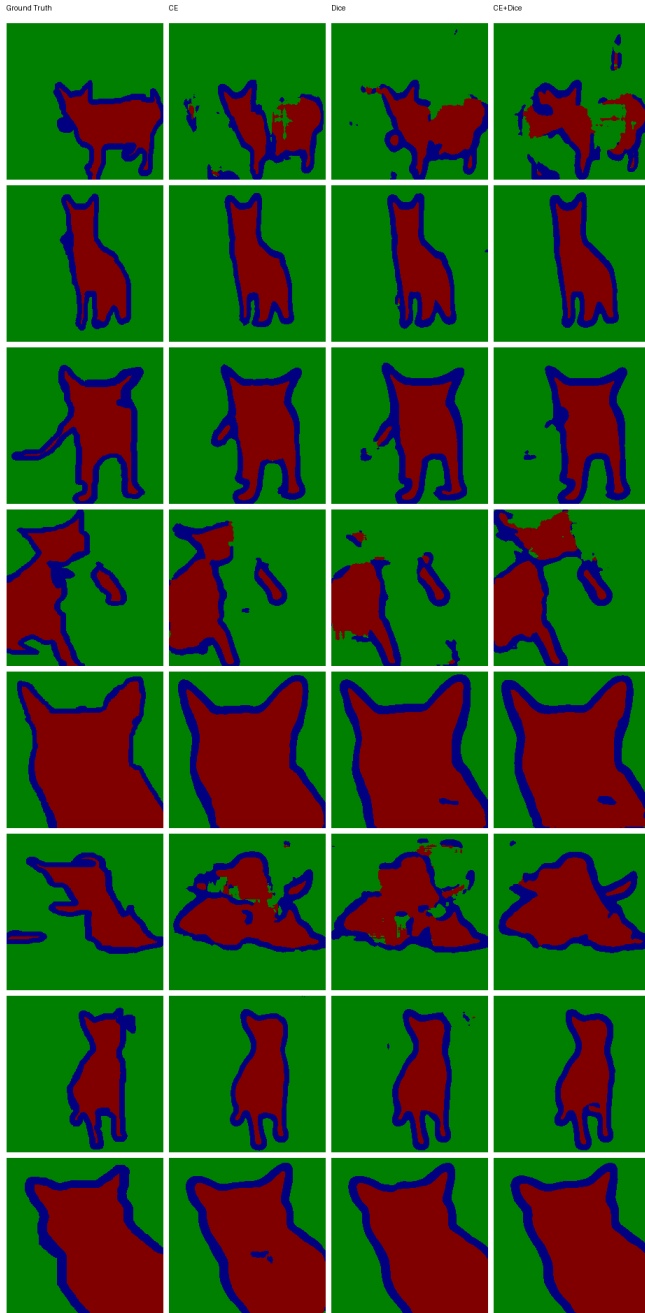


Figure 21: Additional qualitative comparison across the three required segmentation losses. This is a tall diagnostic grid, so it is kept as a single-column float rather than being stretched across both columns. Dice usually produces the cleanest object body, while CE and CE+Dice can differ around boundary pixels.

where $P(R)$ is precision as a function of recall. In practice, modern evaluation uses discrete interpolation over the confidence-ranked detections.

D.2 mAP50 and mAP50-95

mAP50 uses an IoU threshold of 0.50. A detection is correct if the predicted box overlaps the target by at least 50%. This is relatively forgiving and is useful for judging whether the model found the object. mAP50-95 averages AP over thresh-

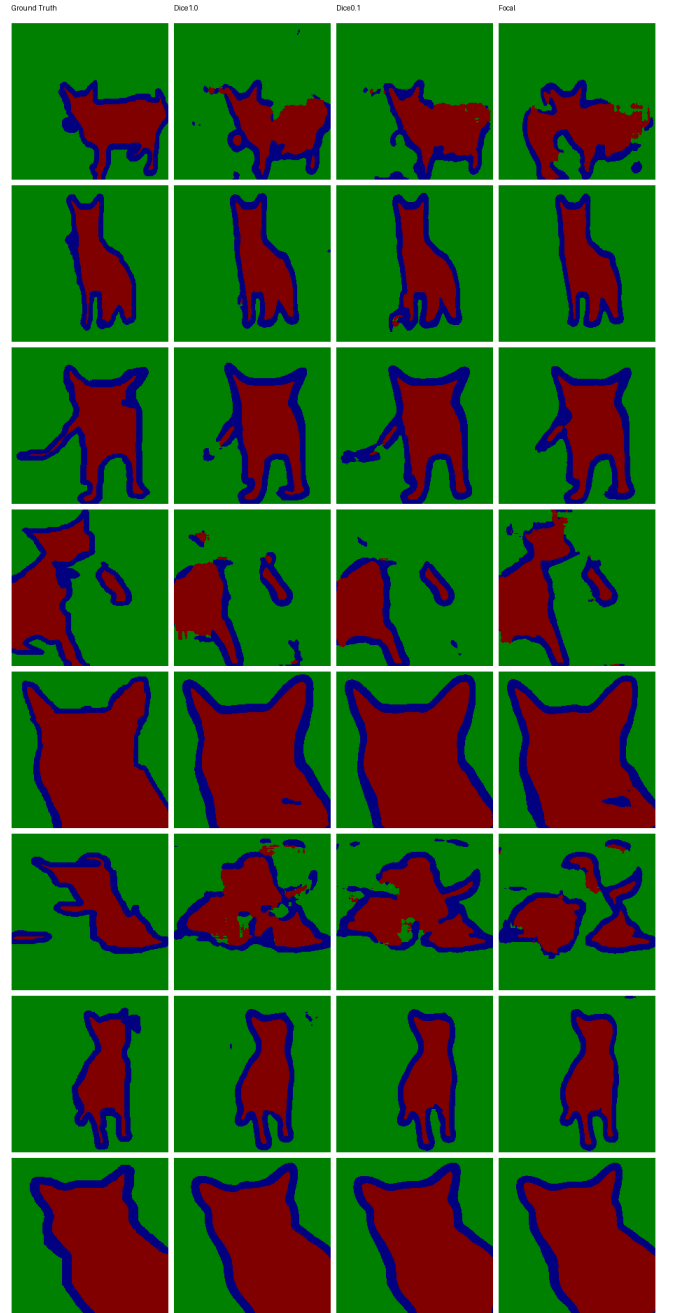


Figure 22: Additional qualitative comparison for segmentation extensions. The two Dice variants are visually similar, consistent with the small numerical difference. Focal Loss does not remove the dominant boundary errors, which supports the interpretation that hard-pixel weighting is not enough for trimap contour refinement.

olds from 0.50 to 0.95, usually in steps of 0.05:

$$mAP_{50-95} = \frac{1}{10} \sum_{t \in \{0.50, 0.55, \dots, 0.95\}} AP_t. \quad (13)$$

This is much stricter because high IoU thresholds require precise localization. Small objects make this especially difficult. If a pedestrian is only a few pixels wide, a small coordinate error can greatly reduce IoU. Therefore, the gap between mAP50 and mAP50-95 is expected on VisDrone.

The YOLO11m result, 0.5356 mAP50 and 0.3310 mAP50-

95, should be read through this lens. The detector often finds objects at a loose IoU threshold, but precise localization remains difficult for tiny, occluded, or dense objects. The improvement over YOLOv8n and YOLOv8s is meaningful because it raises both loose detection quality and stricter localization quality.

D.3 Why Resolution Helps VisDrone

For an ordinary close-range image, a car may occupy hundreds of pixels. In VisDrone, the same class may appear as a small rectangle in a crowded road scene. If the model input is resized to 640, small objects can shrink to only a few feature-map cells. Raising the input size to 960 preserves more object detail before the network downsamples. This does not guarantee perfect detection, but it increases the chance that small targets survive the early convolutional stages.

The downside is computation. Larger images increase memory usage, training time, and inference cost. In this project the available RTX A6000 memory allowed YOLO11m at 960 with batch size 16. This was a practical compromise: strong enough to improve mAP substantially, but still feasible under the deadline.

D.4 Detection Error Sources

The major detection error sources are small scale, inter-class similarity, occlusion, and annotation density. Pedestrian and people classes can be hard to distinguish at distance. Bicycle, motor, and tricycle categories can overlap visually when riders and vehicles are partially hidden. Cars and vans are easier than tiny pedestrians, but dense traffic and roof-view perspective can still cause missed boxes. These failure modes explain why increasing model capacity improves the result but does not make mAP50-95 close to ordinary close-range detection benchmarks.

The detection results should therefore be judged relative to VisDrone difficulty, not relative to simple object detection tasks. A mAP50-95 of 0.3310 is not a high absolute number, but it is a strong improvement over the YOLOv8n baseline in this specific pipeline, and it enables more reliable video tracking than the smaller detectors.

E Appendix E: Tracking Geometry and Counting Reliability

E.1 Signed-Side Line Crossing

The line-crossing algorithm uses a simple geometric test. Let the counting line have endpoints $A = (x_1, y_1)$ and $B = (x_2, y_2)$. For an object center $C = (x, y)$, define the signed side value

$$s(C) = (x_2 - x_1)(y - y_1) - (y_2 - y_1)(x - x_1). \quad (14)$$

This is the 2D cross product between the line direction and the vector from the first line endpoint to the object center. If

$s(C) > 0$, the point is on one side of the line; if $s(C) < 0$, it is on the other side. A crossing is detected when the sign changes between two frames for the same track ID:

$$s(C_{t-1})s(C_t) < 0. \quad (15)$$

The implementation counts each track ID at most once. This prevents one stable object from being counted repeatedly after it crosses the line.

The method is transparent but not perfect. It assumes that the tracked box center is a stable proxy for object position. For large vehicles or partially occluded pedestrians, the bounding-box center can jump even when the physical object has not clearly crossed. It also assumes that the same physical object keeps the same track ID. If the tracker loses the object and creates a new ID, duplicate or missed counts can occur.

E.2 Confidence Threshold Effects

The tracking comparison shows that confidence threshold is not a cosmetic parameter. At confidence 0.50, ByteTrack counts only 5 crossings. At confidence 0.25, it counts 11 crossings. The lower threshold retains weak detections of small, rainy, and partially occluded objects near the line. This improves recall for counting, but it also risks short-lived false tracks. The best threshold for traffic counting is therefore not necessarily the threshold that produces the cleanest-looking single-frame detections.

This explains why the report treats the final count as an algorithmic estimate. Without manually annotated video ground truth, it is not possible to say whether 11 is the exact real-world count. What the experiment does show is that the pipeline behaves consistently: detector strength affects track continuity, tracker choice affects ID association, and confidence threshold affects the number of observable crossings.

E.3 What Would Be Needed for Production Counting

A production traffic-counting system would need several additions beyond this assignment. First, manually annotated video ground truth would be needed to measure count accuracy. Second, direction-aware counting would be needed to separate incoming and outgoing traffic. Third, camera calibration or a homography would help map the image line to a real-world road line. Fourth, re-identification or stronger tracklet stitching would reduce duplicate counting after long occlusions. These additions are beyond the required homework scope, but they clarify the difference between a reproducible academic demonstration and a deployable traffic-measurement system.

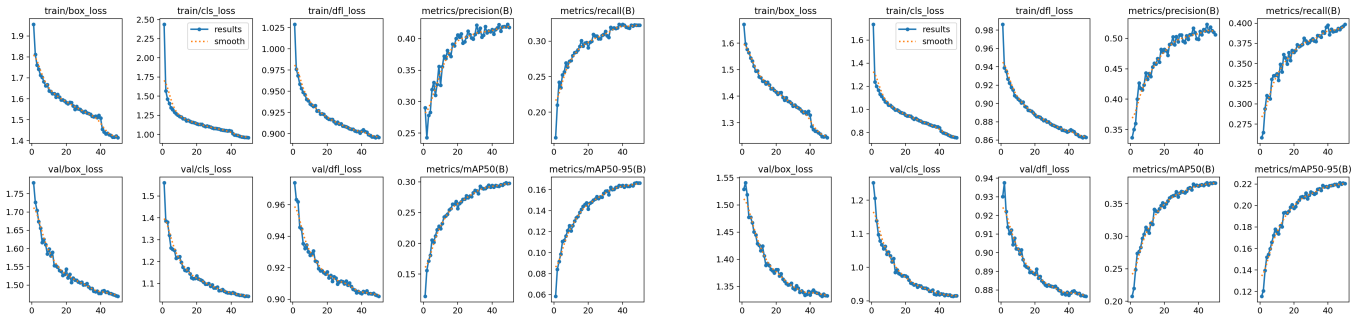


Figure 23: Side-by-side detector training summaries for YOLOv8n and YOLOv8s. The larger YOLOv8s improves the entire metric curve, but recall remains harder than precision because many VisDrone targets are small, occluded, or low contrast.

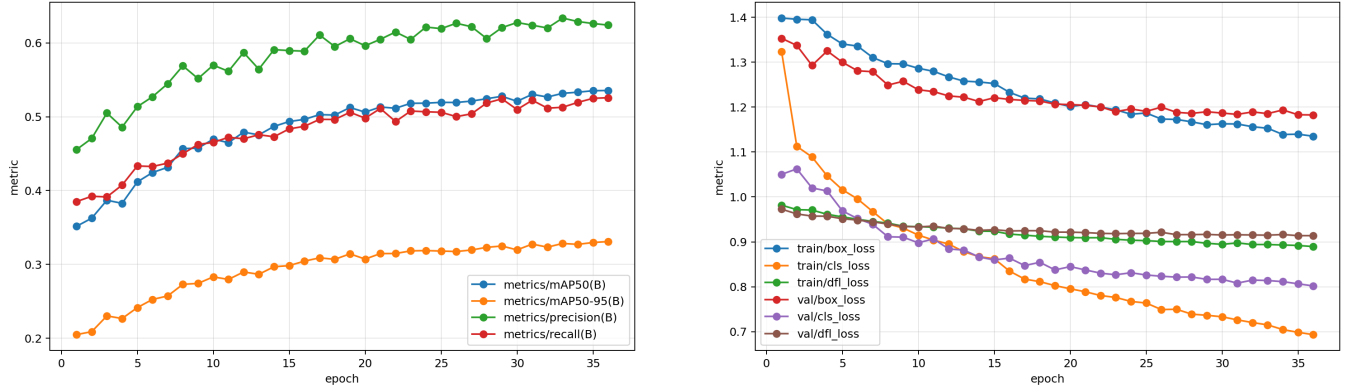


Figure 24: YOLO11m at image size 960. The metric curves and loss curves show that the model was still learning when manually stopped at epoch 36. The reported score is therefore a deadline-constrained result rather than a fully saturated training run.



Figure 25: Preview frames from the uploaded video. The scene contains rain, road reflections, moving vehicles, pedestrians, and mixed traffic. These conditions are suitable for demonstrating tracking and line-crossing behavior because objects move, overlap, and sometimes become partially hidden.



Figure 26: Occlusion-focused tracking frames repeated here for closer appendix analysis. The four frames show why stable tracking is harder than single-frame detection: an object may be detectable in one frame, partly hidden in the next, and then reappear with a different visual context.

F Appendix F: Additional Cross-Task Analysis

F.1 Metric Alignment

The strongest method in each task is the method whose training signal or model capacity best matches the evaluation bottleneck. Classification is evaluated by top-1 breed accuracy, but the dominant difficulty is fine-grained representation. Pre-

training and moderate backbone depth address this directly. Segmentation is evaluated by mIoU, and Dice loss directly optimizes overlap, which explains its advantage over cross-entropy. Detection is evaluated by mAP, and VisDrone is dominated by tiny-object visibility and localization; higher input resolution and a stronger detector address this problem more directly than small-model efficiency.

This metric alignment also explains the negative results. SE attention sounds appropriate for fine-grained classifica-

tion, but it does not guarantee a better representation when the pretrained baseline is already strong. Focal Loss sounds appropriate for hard pixels, but many hard pixels are ambiguous boundaries, so emphasizing them does not necessarily improve region IoU. A high detection confidence threshold sounds cleaner, but it reduces recall and line-crossing counts in rainy traffic video. The experiments therefore support a cautious design principle: a method should be judged by whether it targets the actual bottleneck, not by whether it is generally popular.

F.2 Ablation Strength and Fairness

Not all comparisons in this report have the same causal strength. The classification grid search is a controlled hyperparameter analysis because only two hyperparameters change while the architecture and data protocol remain fixed. The segmentation loss comparison is also relatively controlled because the U-Net and data preprocessing remain fixed while the loss changes. The YOLO11m@960 comparison is less isolated because model family and image size change together. This does not make the result invalid, but it changes the claim. The correct claim is not that YOLO11m alone explains the improvement; the correct claim is that the practical combination of a stronger one-stage detector and higher input resolution is substantially better for this VisDrone pipeline.

This distinction matters in a scientific report. A good extension result should not be overstated. When a comparison is controlled, the conclusion can be narrow and causal. When multiple factors change, the conclusion should be practical rather than causal. I used this rule throughout the discussion to avoid turning the report into a list of unqualified improvements.

F.3 Failure Modes as Evidence

The most informative evidence is not only the best score. Classification mistakes reveal that the model is confused among similar breeds, which supports the fine-grained representation interpretation. Segmentation error maps reveal boundary errors, which supports the Dice and boundary-analysis interpretation. Detection and tracking frames reveal small targets, occlusion, and threshold sensitivity, which support the VisDrone difficulty interpretation. These failure modes connect the numerical results to visual behavior.

In this sense, the report’s negative results are not failures of the project. They are evidence about the problem structure. Random initialization shows that the target dataset is not enough to train a deep classifier from scratch under the assignment budget. SE attention shows that extra modules can overcomplicate a strong pretrained baseline. Focal Loss shows that hard-pixel emphasis is not the same as boundary refinement. High tracking confidence shows that visual cleanliness can reduce counting recall. Each negative result narrows the set of explanations and makes the final conclusions more defensible.

F.4 Recommended Future Experiments

If more compute were available, the most useful classification experiment would be a higher-resolution crop or part-aware model for confusing breed groups. The most useful segmentation experiment would be a boundary-aware auxiliary loss or a higher-resolution U-Net decoder. The most useful detection experiment would be a controlled two-factor study: YOLO11m at 640 versus 960, and YOLOv8s at 640 versus 960, so that model capacity and resolution could be separated. The most useful tracking experiment would be manual annotation of the 30-second video, enabling true count accuracy, ID-switch rate, and direction-specific counting metrics.

These future experiments are deliberately tied to observed errors. They are not generic suggestions. They follow directly from where the current models fail: subtle breed cues, trimap boundaries, tiny aerial targets, and rainy-video track continuity.

G Appendix G: Architecture and Tensor-Dimension Trace

G.1 Classification Forward Path

The classification implementation uses torchvision ResNet backbones and replaces only the final fully connected layer. For the main ResNet-18 baseline, the input tensor has shape $B \times 3 \times 224 \times 224$. The first 7×7 convolution with stride 2 reduces the spatial size to 112×112 , max pooling reduces it again to 56×56 , and the four residual stages then produce 56, 28, 14, and 7 pixel feature maps. Global average pooling collapses the final $B \times 512 \times 7 \times 7$ tensor into $B \times 512$, and the new classifier maps it to $B \times 37$ logits.

This trace explains why the input size is not an arbitrary cosmetic choice. At 224×224 , the final feature grid is only 7×7 . The model must preserve discriminative breed evidence through several downsampling stages. If the image crop loses the face, ears, or coat pattern, the final descriptor cannot recover that information. This is one reason the classification errors are mostly between visually similar breeds rather than between cats and dogs.

The SE extension modifies the forward path after Layer4. It first applies global average pooling to $B \times 512 \times 7 \times 7$, producing one scalar descriptor per channel. A two-layer MLP compresses the 512 channels to 32 hidden units because the reduction ratio is 16, then expands back to 512 and applies a sigmoid. The resulting $B \times 512 \times 1 \times 1$ weights multiply the Layer4 feature map channel by channel. In other words, SE does not add new spatial reasoning; it only learns which channels should be emphasized. This is why the SE result should not be interpreted as a failed attention mechanism in general. It is specifically evidence that channel recalibration after a strong pretrained Layer4 was not the limiting factor in this dataset and training budget.

Table 8: Tensor dimensions for the ResNet-18 classification baseline with batch size B .

Stage	Output shape	Interpretation
Input	$B \times 3 \times 224 \times 224$	normalized RGB image
Conv stem	$B \times 64 \times 112 \times 112$	low-level edges/textures
MaxPool	$B \times 64 \times 56 \times 56$	early spatial compression
Layer1	$B \times 64 \times 56 \times 56$	shallow residual features
Layer2	$B \times 128 \times 28 \times 28$	mid-level texture/parts
Layer3	$B \times 256 \times 14 \times 14$	object-level cues
Layer4	$B \times 512 \times 7 \times 7$	breed-discriminative features
AvgPool	$B \times 512$	global descriptor
FC	$B \times 37$	breed logits

G.2 Segmentation Forward Path

The U-Net is implemented from scratch with base channel count 32. Every encoder block is a DoubleConv module: Conv-BN-ReLU followed by another Conv-BN-ReLU. Max-Pool halves the spatial resolution before each deeper stage. The decoder uses transposed convolutions to upsample and then concatenates the corresponding encoder feature map through a skip connection. For input size 224×224 , all down-sampling and upsampling operations align exactly.

The dimension trace is also useful for interpreting the loss results. Cross-entropy treats each output pixel independently after the final 1×1 head. Dice loss aggregates probabilities over all pixels of each class before computing the overlap ratio. Therefore, even though both losses supervise the same $B \times 3 \times 224 \times 224$ logits, they send different gradient signals through the decoder. CE provides dense local feedback, while Dice provides a global region-level signal. The better mIoU of Dice indicates that the bottleneck was not simply assigning every pixel a class, but forming coherent pet, background, and boundary regions.

G.3 Detection Forward Path at a High Level

The detection models are one-stage YOLO detectors. Unlike two-stage detectors, they do not first propose regions and then classify them in a separate second stage. Instead, the backbone extracts multiscale features, the neck fuses features across scales, and the detection head predicts boxes, objectness-related confidence, and class probabilities directly. For VisDrone, multiscale prediction is essential: large vehicles may be represented on coarse feature maps, while pedestrians and bicycles depend on finer feature maps.

The three detector configurations should be understood as a practical capacity-resolution ladder. YOLOv8n at 640 is the lightweight baseline. YOLOv8s at 640 increases capacity but keeps the same input resolution, so it tests whether a larger model helps under the same image size. YOLO11m at 960 changes both model family/capacity and resolution, so it is not a perfectly isolated ablation. It is an engineering extension: use a stronger current one-stage detector and preserve more pixels for tiny aerial objects. This distinction matters because the report should not claim that architecture alone caused the gain. The defensible claim is that the YOLO11m@960 con-

figuration is the best practical detector trained in this project.

H Appendix H: Learning-Curve Diagnostics

H.1 Reading Curves Instead of Only Final Scores

The report uses learning curves as evidence for optimization behavior. A validation curve that peaks early and then flattens suggests that the last epoch is not necessarily the best model. A training curve that continues improving while validation stagnates suggests overfitting or insufficient regularization. A curve where both training and validation remain low suggests underfitting or insufficient representation. This is why the harness always saves the best validation checkpoint and why the report distinguishes best epoch from final epoch.

In classification, the random ResNet-18 is the clearest underfitting case. The model learns from the training set, but validation accuracy remains far below pretrained models. The SE model does not show a catastrophic optimization failure; it simply does not improve generalization. This difference is important. A failed optimization run would suggest a training bug or bad learning rate, while a stable but non-improving run suggests that the modification is not targeting the bottleneck.

The grid search also clarifies a methodological issue. Hyperparameter tuning can easily overfit the validation set if too many combinations are tried. I therefore used a small and interpretable 3×3 grid rather than a large open-ended search. The result is useful even though it does not beat ResNet-34: it demonstrates that the baseline learning-rate and label-smoothing choices were not arbitrary, and it shows that modest smoothing can be beneficial when the model is already pretrained.

H.2 Segmentation Curves and Loss Behavior

The segmentation curves are more subtle than classification curves because loss values are not directly comparable across CE, Dice, and Focal objectives. A lower Dice loss and a lower CE loss are not on the same numerical scale. Therefore, mIoU is the shared comparison axis. This is also why the report plots

Table 9: Tensor dimensions for the implemented U-Net. The concatenation step is the key difference from a plain encoder-decoder: high-resolution encoder features are reused by the decoder.

Stage	Operation	Output shape	Role
Input	image tensor	$B \times 3 \times 224 \times 224$	RGB pet image
Enc1	DoubleConv	$B \times 32 \times 224 \times 224$	fine edges and color
Enc2	Pool + DoubleConv	$B \times 64 \times 112 \times 112$	local parts
Enc3	Pool + DoubleConv	$B \times 128 \times 56 \times 56$	larger body regions
Enc4	Pool + DoubleConv	$B \times 256 \times 28 \times 28$	coarse object structure
Bottleneck	Pool + DoubleConv	$B \times 512 \times 14 \times 14$	global context
Dec4	Up + concat + DoubleConv	$B \times 256 \times 28 \times 28$	recover coarse mask
Dec3	Up + concat + DoubleConv	$B \times 128 \times 56 \times 56$	refine object layout
Dec2	Up + concat + DoubleConv	$B \times 64 \times 112 \times 112$	refine boundaries
Dec1	Up + concat + DoubleConv	$B \times 32 \times 224 \times 224$	pixel-level details
Head	1×1 convolution	$B \times 3 \times 224 \times 224$	trimap logits

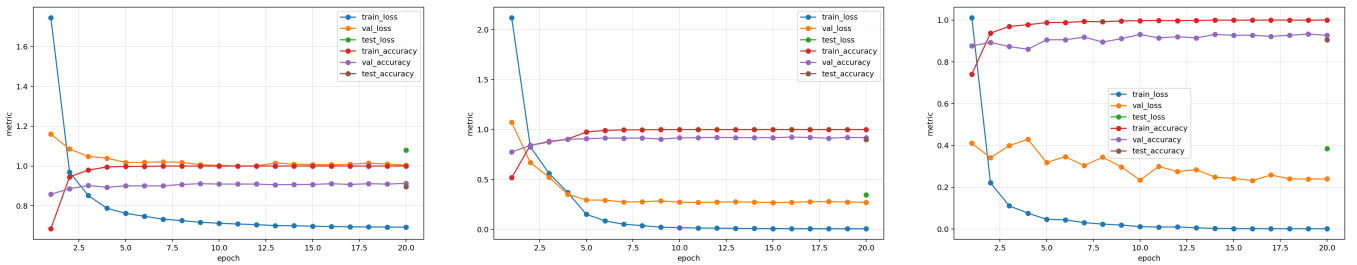


Figure 27: Additional classification extension curves. Label smoothing mainly regularizes confidence, freeze-then-unfreeze stabilizes early optimization by first training the new head, and ResNet-34 improves capacity while preserving ImageNet transfer. These curves explain why the final ranking is not determined by one isolated epoch.

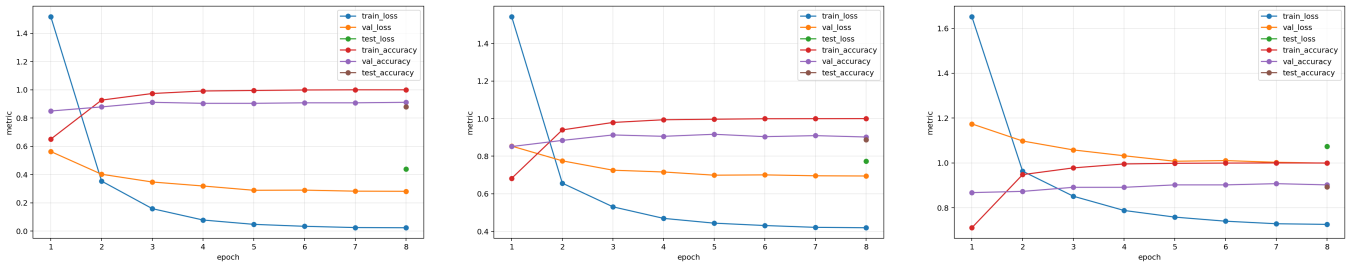


Figure 28: Representative curves from the classification hyperparameter grid. The middle setting, label smoothing 0.05 and head learning rate 10^{-3} , was selected by validation accuracy. The low-head-LR setting adapts the classifier more slowly, while the high-smoothing/high-LR setting does not produce a reliable improvement.

test mIoU summaries separately from loss curves.

The extension result is deliberately interpreted conservatively. Smooth Dice 0.1 gives the highest validation mIoU, but Dice with the default smoothing gives the highest test mIoU. These two results are close enough that they should not be described as a decisive superiority of one smoothing constant. A more defensible interpretation is that Dice-style overlap supervision is robustly better than pure CE, while the exact smoothing constant is a second-order detail under this setup.

I Appendix I: Additional Visual Evidence

I.1 Segmentation Prediction Examples

The following masks show individual predictions rather than pre-arranged comparison grids. They are useful because they reveal a repeated error pattern: the object body is usually well segmented, while thin structures and boundaries remain difficult. This supports the earlier mathematical argument that mIoU rewards overlap but still penalizes boundary mismatch, especially for the boundary class.

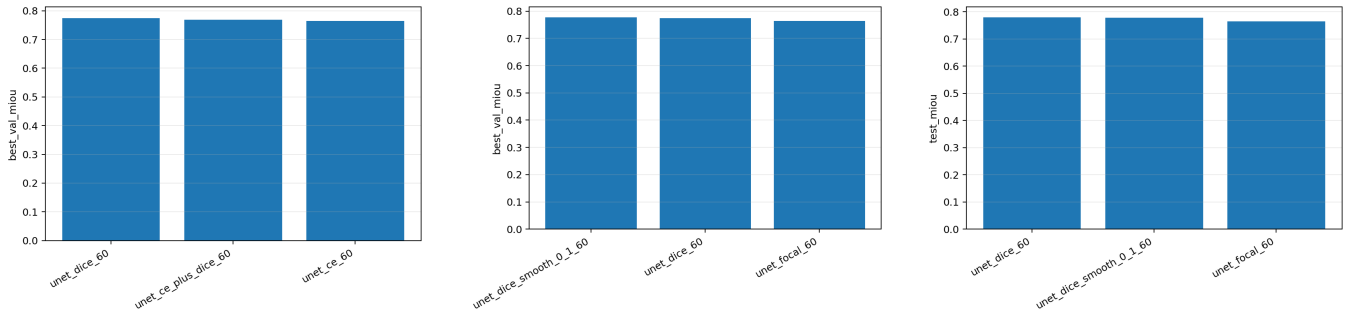


Figure 29: Segmentation metric summaries. Dice is the best required loss, and the smooth-0.1 extension slightly improves validation mIoU but not test mIoU. This is a useful example where validation improvement is real but not large enough to change the final practical conclusion.

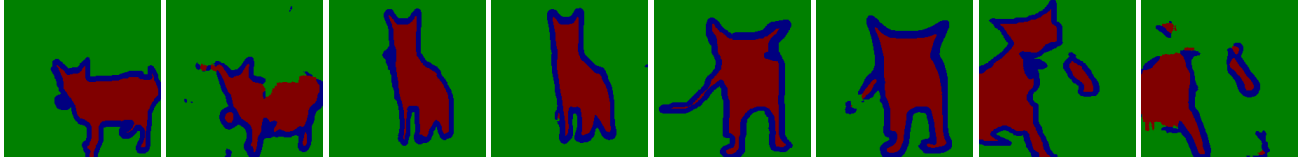


Figure 30: Additional Dice-U-Net target/prediction mask pairs. Each adjacent pair shows target then prediction. The compact layout keeps the masks within the page width while allowing several examples to be compared at once.

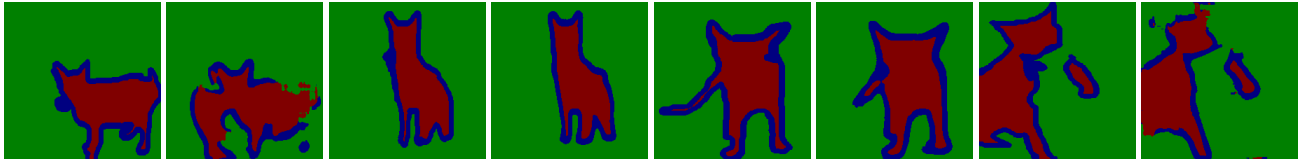


Figure 31: Additional Focal-U-Net target/prediction mask pairs. Focal Loss changes which pixels receive stronger gradients, but the dominant boundary ambiguity remains visible.

I.2 Detection PR Curves and Confusion Matrix

The YOLOv8s diagnostic plots help explain the baseline extension before YOLO11m. The precision-recall curve summarizes how the detector behaves as the confidence threshold changes. A model with high precision at low recall can be visually clean but miss many objects. A model with higher recall but low precision may overproduce boxes and harm tracking. For traffic counting, the useful operating point is not necessarily the one with the highest single-frame visual neatness; it is the point that preserves enough true objects for stable tracks without creating too many false trajectories.

The normalized confusion matrix should not be read like a classification confusion matrix with one object per image. Detection errors combine missed objects, duplicate boxes, class confusion, and localization failure. For small road users, class errors can occur because the visible evidence is tiny: a pedestrian, rider, bicycle, and motorbike may occupy only a small cluster of pixels. This again supports the choice of higher resolution for the strongest detector.

I.3 Tracking Frames and Counting Interpretation

Line-crossing counting is sensitive to frame-to-frame continuity. A single missed box may not matter if the tracker bridges it, but repeated misses near the counting line can prevent a

crossing from being recorded. Conversely, a false short track that crosses the line can inflate the count. Therefore, the reported count should be interpreted as a reproducible pipeline estimate, not as manually verified ground truth.

J Appendix J: Metric Derivations and Practical Consequences

J.1 Why Dice Aligns Better with mIoU

For one class, Dice and IoU are monotonically related by $Dice = 2IoU / (1 + IoU)$. Taking the derivative with respect to IoU gives

$$\frac{dDice}{dIoU} = \frac{2}{(1 + IoU)^2} > 0. \quad (16)$$

Thus, any improvement in IoU improves Dice, although the slope decreases as IoU becomes high. This derivative explains why Dice loss is especially useful when overlap is still moderate: it gives strong feedback for improving region overlap early and in the middle of training. Cross-entropy does not have this direct overlap geometry. It can reduce loss by becoming more confident on many easy pixels even if the predicted object boundary does not align much better with the ground truth.

For multiclass segmentation, the implementation computes softmax probabilities, converts the target to one-hot maps,

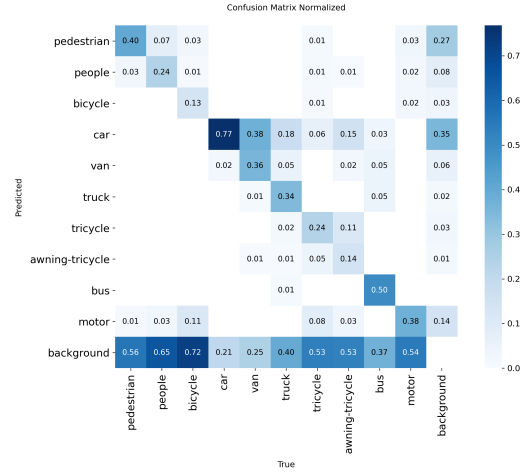
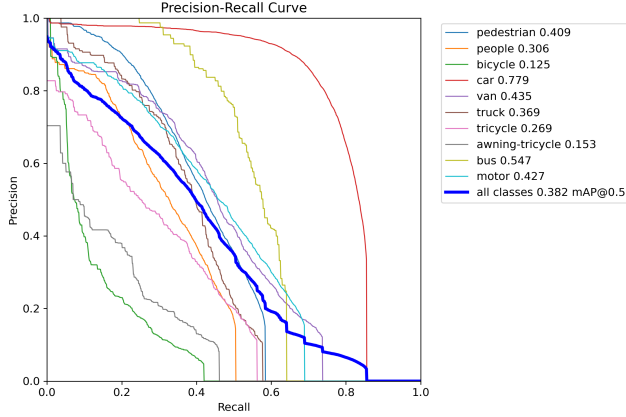


Figure 32: YOLOv8s precision-recall curve and normalized confusion matrix. The PR curve shows the threshold trade-off, while the confusion matrix highlights class-level ambiguity in dense aerial scenes.



Figure 33: Additional crossing-focused frames. The sequence shows why counting depends on both detector confidence and ID continuity: the object must remain detected and associated while its center moves across the virtual line.



Figure 34: Five preview frames across the source video. The visual conditions change across time because of rain, reflections, object density, and camera viewpoint. This justifies evaluating tracking over a 30-second segment rather than a single attractive frame.

computes class-wise Dice, and averages over classes. This has two practical effects. First, every class contributes to the loss even if its pixel count is smaller. Second, the loss sees the predicted probability mass over the whole image, not only independent pixel decisions. These properties are well matched to mIoU, which also averages region overlap across classes.

J.2 How mAP Penalizes Ranking and Localization

Average Precision is not simply the precision at one confidence threshold. It evaluates the whole ranked list of detections. Suppose predictions are sorted from highest to lowest confidence. As the threshold moves down this list, recall usually increases because more boxes are accepted, while precision may decrease because false positives enter the set. AP summarizes the area under this precision-recall behavior. A detector can therefore lose AP in two ways: it may fail to detect objects at all, or it may rank false detections above true detections.

The IoU threshold adds another layer. At mAP50, a roughly

correct box can count as correct. At mAP75 or mAP95, the same box may become a false positive if its localization is slightly off. In aerial images, small boxes make this severe. Moving a box boundary by only a few pixels can change IoU substantially. This is why YOLO11m’s mAP50 of 0.5356 and mAP50-95 of 0.3310 should be reported together. The first says the model often finds the object; the second says precise localization across strict thresholds remains hard.

J.3 Why Test Results Should Not Guide Model Selection

The harness keeps validation and test roles separate. Validation is used repeatedly during development: choosing best epoch, selecting hyperparameters, and comparing extensions. Test is used after the choice is made. If the test set were used to choose label smoothing, learning rate, or the best checkpoint, the reported test accuracy would no longer be an unbiased estimate of generalization. This distinction is especially important in this project because many experiments were run. More experiments increase the chance that one setting performs well

by chance on a particular split.

This is also why some appendix analysis is deliberately cautious. For example, the classification grid identifies a good ResNet-18 hyperparameter setting, but the final classification conclusion still favors ResNet-34 because it has the best held-out test result among trained models. For segmentation, smooth Dice has the best validation mIoU but not the best test mIoU, so the conclusion emphasizes the robust advantage of Dice-style loss rather than claiming that smooth 0.1 is universally better. For detection, YOLO11m@960 is the strongest practical configuration, but the analysis does not isolate whether the gain comes more from architecture, capacity, or input size.

K Appendix K: Limitations and Threats to Validity

K.1 Compute Budget and Early Stopping

The detection run was manually truncated at epoch 36 because of time constraints. The curves suggested that additional training might still improve the result. This does not invalidate the comparison, because the recorded YOLO11m score already exceeds YOLOv8n and YOLOv8s substantially, but it does limit the strength of claims about final convergence. The correct statement is that YOLO11m@960 was the strongest model obtained under the project budget, not necessarily the strongest score that the configuration could ever reach.

The classification and segmentation experiments are less affected by this limitation because they completed their planned budgets and used validation-best checkpoints. However, even there, compute budget shapes the conclusions. Random ResNet-18 might improve with much longer training, but the assignment-relevant conclusion is about the realistic homework budget. Under that budget, pretraining is clearly necessary for high accuracy.

K.2 Dataset and Annotation Limitations

Oxford-IIIT Pet classification labels are breed labels, so the task rewards fine-grained appearance rather than coarse object presence. Some mistakes may be understandable even if the predicted class is visually similar to the label. The segmentation trimaps contain a boundary class that is intrinsically ambiguous after resizing. VisDrone annotations are dense and small, and the video tracking clip was not manually annotated frame by frame. These dataset properties affect what the metrics mean.

For this reason, the report combines scalar metrics with qualitative figures. A single number cannot reveal whether errors are random, boundary-focused, class-confusion-focused, or caused by occlusion. The visual evidence shows that the main errors are structured. Classification errors occur among similar breeds, segmentation errors concentrate around boundaries, detection errors occur for small and occluded objects,

and tracking errors are tied to ID continuity. Structured errors are more scientifically useful than unexplained low scores because they indicate where future work should focus.

K.3 Reproducibility Versus Exhaustiveness

The project prioritizes reproducible harness-based experimentation over an exhaustive search of every possible model. This trade-off is intentional. A large collection of uncontrolled experiments would make the report longer but less convincing. Instead, each required task has a clear baseline, a controlled or partially controlled extension, saved logs, saved checkpoints, and visual outputs. The appendix adds details not to inflate the result, but to expose the reasoning behind the conclusions.

The main remaining threat to validity is that some extensions change more than one factor. The YOLO11m@960 experiment changes both model and resolution. The ResNet-34 experiment changes depth while preserving the pretrained ResNet family. These are still useful engineering comparisons, but they should be labelled as practical improvements rather than pure causal ablations. This distinction is now reflected explicitly in the discussion.

L Appendix L: Quantitative Delta Analysis

L.1 Absolute and Relative Improvements

Raw scores are necessary, but score deltas make the experimental conclusions easier to compare. For a baseline score s_b and an improved score s_i , the absolute improvement is

$$\Delta_{abs} = s_i - s_b, \quad (17)$$

and the relative improvement over the baseline is

$$\Delta_{rel} = \frac{s_i - s_b}{s_b}. \quad (18)$$

The absolute number is usually more meaningful for final reporting because metrics such as accuracy, mIoU, and mAP already live on a bounded $[0, 1]$ scale. The relative number is useful for understanding how much the baseline error or weakness has been reduced, but it can exaggerate improvements when the baseline is very small.

The classification delta between random and pretrained ResNet-18 is much larger than the deltas among pretrained variants. This is the signature of a representation bottleneck. Once ImageNet features are available, the model already has useful edge, texture, part, and object-level filters, so later improvements are incremental. ResNet-34 improves the result because it adds depth while retaining pretraining, but the improvement is not comparable to the pretraining jump.

The segmentation deltas are smaller in absolute terms, but this does not mean they are unimportant. mIoU is a strict region-overlap metric averaged over classes. A 1.5-point gain

Table 10: Selected score deltas across tasks. The numbers show that the largest project-level gains came from matching the method to the dominant bottleneck: pretraining for classification, overlap loss for segmentation, and stronger high-resolution detection for VisDrone.

Task	Comparison	Metric	Absolute delta	Interpretation
Classification	ResNet-18 pretrained vs random	test acc.	+0.4925	transfer learning dominates
Classification	ResNet-34 vs ResNet-18	test acc.	+0.0120	moderate capacity gain
Classification	grid best vs baseline	val acc.	−0.0018	grid confirms baseline is strong
Segmentation	Dice vs CE	test mIoU	+0.0150	overlap objective helps
Segmentation	Smooth Dice vs Dice	test mIoU	−0.0005	smoothing effect is minor
Detection	YOLOv8s vs YOLOv8n	mAP50-95	+0.0546	capacity helps at 640
Detection	YOLO11m@960 vs YOLOv8s	mAP50-95	+0.1099	capacity plus resolution helps more

can reflect many corrected pixels, especially in the thin boundary class. The small difference between default Dice and smooth Dice means that the broad choice of overlap-based supervision matters more than the exact smoothing constant.

The detection deltas are the largest among the model-extension results after classification pretraining. YOLOv8s improves over YOLOv8n at the same input size, so capacity clearly helps. YOLO11m@960 improves much more, which supports the tiny-object argument. The result is practically useful for the video pipeline because more correct detections increase the chance that ByteTrack can maintain identities through motion and partial occlusion.

L.2 Why Some Small Gains Are Still Worth Reporting

In homework reports, it is tempting to hide small or negative gains. I kept them because they make the study more credible. The SE-ResNet-18 extension did not beat the baseline, but it shows that channel attention was not the missing component under this protocol. Smooth Dice did not beat default Dice on the test set, but it shows that the Dice family is stable and that the exact smoothing constant is not the main source of performance. High-confidence tracking produced fewer counted crossings, but it shows that visually cleaner detections can be worse for counting recall.

This is a scientific advantage rather than a weakness. A report that only includes winning settings can create a false impression that every popular technique works. The actual experiments show a more useful rule: improvements happen when the modification addresses the task bottleneck. Pre-training addresses limited classification data; Dice addresses overlap-based segmentation evaluation; higher-resolution detection addresses small aerial targets. SE attention, focal hard-pixel emphasis, and high confidence thresholds are not wrong ideas, but they do not directly solve the observed bottlenecks.

M Appendix M: Additional Detection Diagnostics

M.1 Confidence, Recall, and Tracking

For object detection alone, raising the confidence threshold often makes visual outputs cleaner because weak boxes dis-

appear. For tracking and counting, the same change can be harmful. ByteTrack is designed to use lower-confidence detections to recover track continuity. If the detector suppresses too many weak but real boxes, tracks break near occlusions or near the counting line. The final count therefore depends on the detector and tracker as a coupled system.

The tracking results are therefore not merely a post-processing add-on. They test whether the detector’s errors are tolerable over time. A false negative in one frame may be bridged by a tracker, but repeated false negatives break the ID. A false positive that lasts only one frame may be harmless, but a false track that crosses the line may inflate the count. The best operating point is a compromise between single-frame detection quality and temporal continuity.

M.2 Why mAP and Counting Can Disagree

mAP is computed on annotated still images. Counting is computed on a video with a virtual line. These are related but not identical objectives. A detector with higher mAP usually helps counting, but the final count also depends on camera geometry, motion direction, tracker ID switches, and whether missed detections happen near the line. A missed object far away from the line may not change the count; a missed object exactly while crossing can change the count. Conversely, a false positive far from the line may not matter, while a false track crossing the line does.

This distinction explains why the report presents both detector validation metrics and video-frame evidence. The detector metrics establish that YOLO11m@960 is the strongest trained model on VisDrone. The video experiment then demonstrates how that detector behaves in the downstream setting. The two parts answer different questions: “Can the model detect objects in the dataset?” and “Can the trained detector support a concrete counting pipeline?”

M.3 Manual Video Choice and External Validity

The selected video segment is rainy and visually difficult. This is useful for demonstration because a trivial clean video would not reveal occlusion, threshold sensitivity, or ID continuity issues. However, it also means that the counted number is not a universal performance estimate. A sunny road scene with larger vehicles would likely produce more stable tracks. A

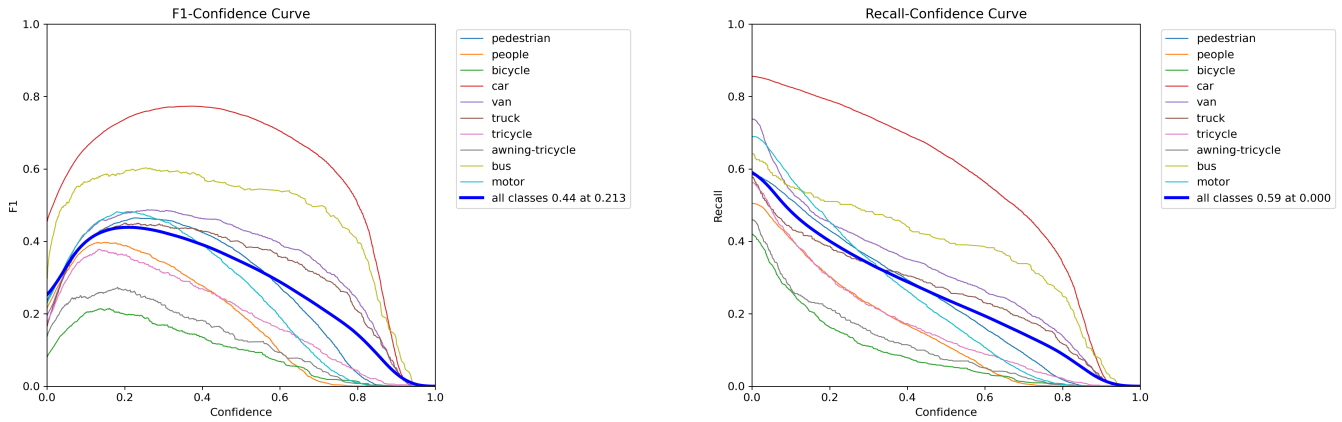


Figure 35: Additional YOLOv8s confidence-diagnostic curves. F1 combines precision and recall at different confidence levels, while the recall curve shows how quickly objects disappear as the threshold becomes stricter. This is directly relevant to the later line-crossing count.

denser crowd or stronger camera motion would likely produce more ID switches. The video is therefore best interpreted as a stress test for the trained detector and tracker rather than a full benchmark.

The assignment requirement is satisfied because the detector is trained on the required VisDrone aerial detection dataset, and the video stage uses the trained detector for a real tracking/counting task. The video does not replace VisDrone training; it extends the trained detector into a temporal application. This distinction is important: the dataset supplies supervised detection learning, while the video supplies qualitative and algorithmic evaluation of deployment behavior.

N Appendix N: What I Would Say in an Oral Defense

N.1 Classification Defense

If asked why the classification model works, the short answer is transfer learning. The input image is resized and normalized so that it matches the statistical assumptions of the ImageNet-pretrained ResNet. The convolutional stem extracts low-level patterns, residual blocks build deeper visual features, global average pooling forms one image-level descriptor, and the new fully connected layer maps that descriptor to 37 breed classes. The experiment shows that this pretrained representation is much more important than adding an SE block. The strongest extension, ResNet-34, helps because it increases depth without abandoning the pretrained feature hierarchy.

If asked why label smoothing is useful, the answer is calibration and regularization. Ordinary cross-entropy treats the correct class as probability 1 and all other classes as probability 0. Label smoothing softens this target distribution, so the model is not rewarded for becoming infinitely confident. In fine-grained breed recognition, some classes are visually similar, so overconfidence can hurt generalization. The grid search shows that mild smoothing can be reasonable, but too much smoothing or too high a head learning rate does not produce a decisive gain.

N.2 Segmentation Defense

If asked why Dice loss fits segmentation, the key point is that the evaluation metric is overlap-based. Cross-entropy supervises each pixel separately. Dice loss compares the predicted and true regions as sets through their intersection and total size. Since Dice is monotonically related to IoU for a single class, it naturally aligns with mIoU better than pure pixel-wise CE. The result confirms this: Dice gives the best required segmentation test mIoU.

If asked why boundaries remain difficult, the answer is annotation and resolution. The trimap boundary class is thin, and after resizing to 224×224 , fur edges and small structures become compressed. The U-Net skip connections help recover spatial detail, but they cannot perfectly reconstruct ambiguous boundaries. This is why future improvements should target boundary-aware losses, higher resolution, or decoder deep supervision instead of only changing the global loss.

N.3 Detection and Tracking Defense

If asked why mAP50 and mAP50-95 differ so much, the answer is localization strictness. mAP50 only requires IoU at least 0.5, so a roughly correct box can count. mAP50-95 averages over stricter thresholds up to 0.95, where even small coordinate errors matter. VisDrone objects are tiny, so a few pixels of error can strongly affect IoU. Therefore, the gap between 0.5356 mAP50 and 0.3310 mAP50-95 is expected and should not be mistaken for a bug.

If asked why the final detector uses YOLO11m@960, the answer is practical bottleneck matching. YOLOv8n is fast but misses many small targets. YOLOv8s improves capacity at the same 640 resolution. YOLO11m@960 preserves more pixels and uses a stronger one-stage detector, giving the best mAP and the most useful tracking input. The comparison is not a pure architecture ablation because resolution also changes, but it is a valid engineering extension for a difficult aerial dataset.

O Appendix O: Submission Artifacts and Layout Policy

O.1 Model Artifacts

The final model weights are hosted externally on ModelScope at <https://www.modelscope.cn/models/yuhengli/cv-hw2-final-model>. This separation is intentional. The GitHub repository should contain source code, configuration files, scripts, the report, and lightweight visual artifacts, while large neural-network checkpoints are better stored in a model repository. This prevents the code repository from becoming unnecessarily large and makes the final trained models easier to download independently.

The artifact set contains the final models used for the three tasks and the comparison checkpoints needed to support the report. For classification, the important checkpoints are the pretrained ResNet-18 baseline, the random-initialization ablation, the SE-ResNet-18 extension, and the stronger ResNet-34 extension. For segmentation, the important checkpoints are U-Net with CE, Dice, CE+Dice, smooth Dice, and Focal Loss. For detection, the important checkpoints are the YOLOv8n baseline, the YOLOv8s capacity extension, and the final YOLO11m@960 model used for video tracking.

O.2 Why the Report Keeps Code, Weights, and Figures Separate

The report is not meant to be a replacement for the repository. It explains the experimental logic, gives the most important results, and includes representative visual evidence. The repository stores the exact commands, configs, and scripts that generated those results. The ModelScope page stores the heavy binary weights. Keeping these roles separate improves reproducibility: a reader can inspect the paper for reasoning, inspect GitHub for implementation, and download ModelScope weights for re-running inference.

This organization also reduces ambiguity in grading. If a scalar result appears in the paper, it should be traceable to a saved summary file or training log. If a figure appears in the paper, it should be traceable to an asset generated by the harness. If a trained model is discussed as final, it should be available as an external checkpoint. The project was organized around this traceability principle from the beginning.

O.3 Figure Layout Policy

Because the report uses a two-column format, figures are placed according to their geometry. Narrow or tall figures use ordinary single-column floats with width no larger than `\linewidth` and height constrained when necessary. This prevents tall grids from overflowing the page and lets the surrounding text remain in two columns. Wide comparison figures use `figure*` so that they may span both columns; after the caption, the document returns to the normal two-column flow automatically.

Captions are placed below every figure, following standard paper style. I avoided manually forcing most figures with the `H` placement specifier because hard placement often creates large blank regions in two-column papers. Instead, the report relies on LaTeX floats with conservative scaling. This is the reason some figures may appear slightly after the paragraph that first discusses them: the trade-off is better page safety and fewer overfull graphics.

The final layout therefore follows a practical rule. If the reader needs side-by-side comparison, the figure is allowed to span two columns. If the figure is only a diagnostic image or a tall grid, it stays in one column. This makes the report denser and prevents a single image from breaking the surrounding text flow.

O.4 Command-Level Reproducibility

The experiments are also reproducible at the command level. All training entry points are exposed through the same package interface, `python -m cvhw2`, and each command receives a YAML configuration file. This design keeps command lines short while moving experimental variables into version-controlled configs. It also reduces the chance of accidental differences between runs because the run directory, checkpoint directory, seed, optimizer settings, and data paths are recorded together.

This command structure is part of the harness contribution. In a manual notebook workflow, it is easy to forget which learning rate, checkpoint, or split was used to create a figure. In this project, every important run has a named config and a corresponding output directory. The report can therefore cite figures and numbers with less ambiguity. If a reader wants to challenge a result, the correct response is not a vague description of training, but a concrete config, checkpoint, log file, and command path.

The same principle applies to failed or weaker experiments. Random ResNet-18, SE-ResNet-18, Focal U-Net, and high-confidence tracking are not discarded. They remain reproducible runs with saved outputs. This matters because negative results are only useful if they can be inspected. A weak result without logs is merely anecdotal; a weak result with a config, curve, checkpoint, and visualization is evidence about the task.

References

- [1] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and Dogs. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2012.
- [2] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [3] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. In *MICCAI*, 2015.

Table 11: Command-level reproducibility map. All entries use the common prefix `PYTHONPATH=src python -m cvhw2`; the exact config file is the unit of experiment definition.

Task	Command pattern	Example config role
Classification	<code>train-classification --config <cfg></code>	ResNet-18, SE-ResNet-18, ResNet-34, grid search
Segmentation	<code>train-segmentation --config <cfg></code>	CE, Dice, CE+Dice, smooth Dice, Focal
Detection	<code>train-detection --config <cfg></code>	YOLOv8n, YOLOv8s, YOLO11m@960
Analysis	<code>analyze-classification --config <cfg></code>	confusion matrix and mistake grids
Tracking	<code>track-video --weights <pt></code>	ByteTrack/BoT-SORT video counting

- [4] P. Zhu et al. Vision Meets Drones: A Challenge. *arXiv:1804.07437*, 2018.
- [5] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLO. <https://github.com/ultralytics/ultralytics>, 2023.
- [6] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. In *ECCV*, 2022.